

시뮬레이션 2조

Term Project Stage2

PC방 좌석 레이아웃 개선 시뮬레이션

202327503 권나영(분석, 발표)
202127504 김동욱(개발자)
202127507 김민구(분석, 발표)
202127510 김범준(개발자)

202327524 박시윤(분석)
202227550 보양바트 벌러러(분석)
202127526 안형권(개발자) – 팀장
202127534 이현진(개발자)

목차

01 | 대상 시스템 소개

02 | 시뮬레이션 모델링

03 | Verification & Validation

04 | 생산성 분석

05 | 결론 및 요약, 한계 및 향후 개선 방향

01. 대상 시스템 소개

2

■ 문제 상황 정의

단체로 게임을 하기 위해 PC방에 방문하는 경우
같이 간 인원만큼의 **연속 좌석** 선호



현재 PC방 점유율 1위 게임인 리그 오브 레전드의
파티원 수인 **5인, 3인** 선호

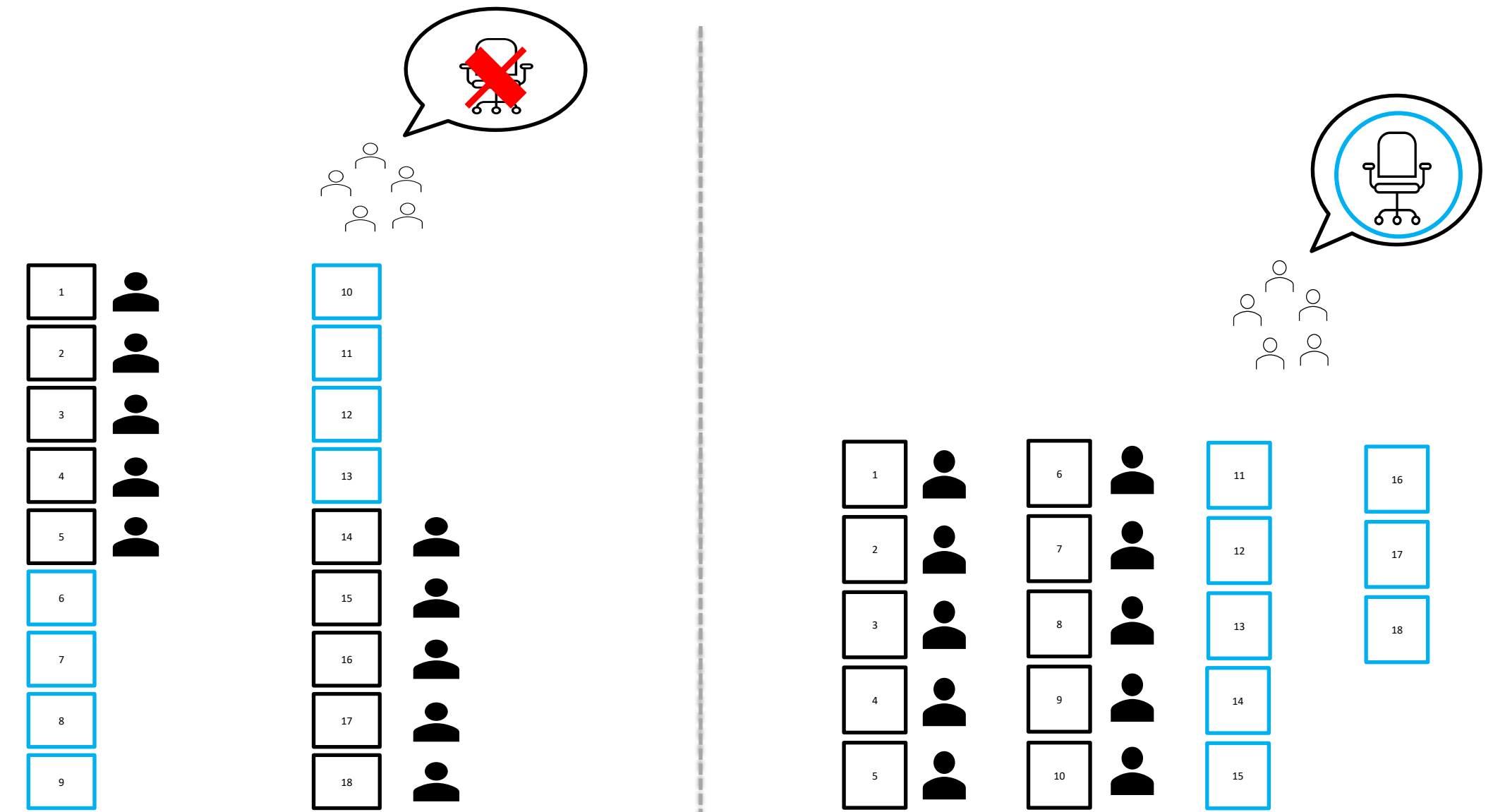


현재 아테나 PC방의 경우, **4인석**이 많이 분포
→ 자리가 없는 경우 다른 PC방으로 이동



다른 PC방으로의 이동 = 수요의 상실 → 수익 감소

■ 주제 선정



<문제 상황>

손님의 수요 분포를 추정하고,
단체 손님을 최대한으로 수용하는 **PC방 레이아웃**을
설계하여 수요의 상실을 방지하는 것이 목표!

01. 대상 시스템 소개

■ 시뮬레이션이 필요한 이유

1. 복잡한 현실 시스템의 효과적인 반영 및 분석

- **수요의 불확실성** 반영: 손님의 도착 간격, 체류시간의 차이, 단체 손님의 유형별 방문 수 등 현실적인 변동 요인을 반영하여 분석 가능
- 복잡한 상호작용 실험: 손님의 행동 원칙을 모델링하여 시스템 내부의 복잡한 상호작용 실험 가능

2. 비용 & 리스크가 큰 실제 실험 대신 최적의 레이아웃 도출

- 실제 변경 없이 다양한 시나리오 검토: 기존 레이아웃을 유지하면서, 다양한 레이아웃 시나리오를 구성 가능^[1]
- **손실 방지**를 위한 예측: 단체 손님 수용을 극대화하는 레이아웃을 설계하여, 좌석 부족으로 인한 수요 상실을 사전에 예측하고 방지 가능

3. 시스템 운영 변수의 종합적인 영향 조사 및 분석

- **KPI를 통한 정량적 비교 분석**: 레이아웃별 좌석 활용률, 이탈 인원(이탈률), 총 매출 등의 KPI를 계산하고 비교 가능
- 운영 변수 확장 가능성: 초기 레이아웃 분석 외에도 음식 주문 시스템, 점원 수, 점원의 이동 경로 등 추가적인 운영 변수까지 고려 가능^[2]

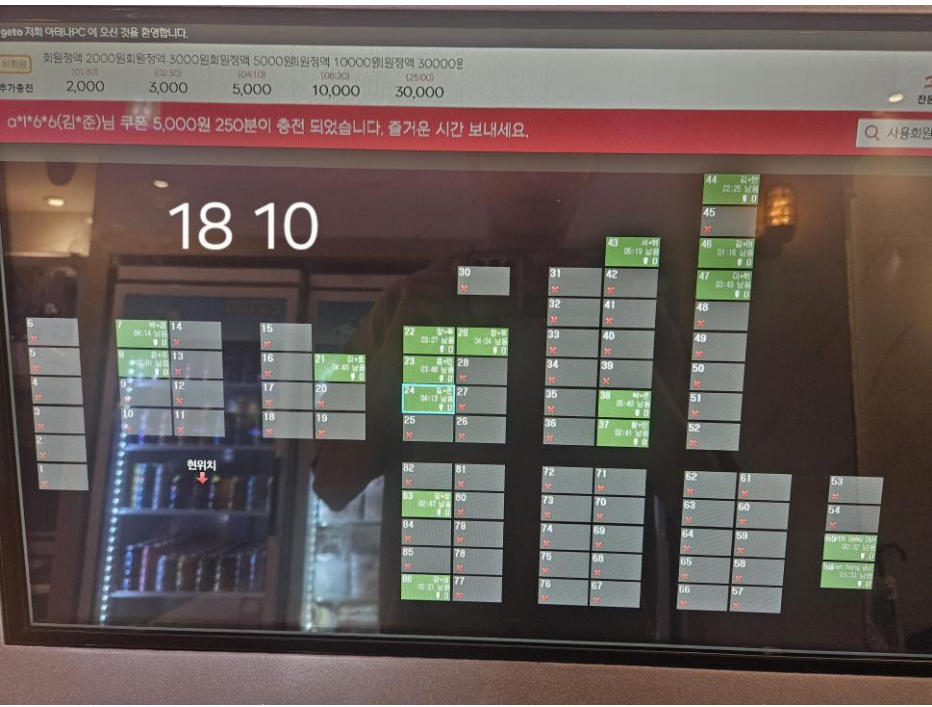
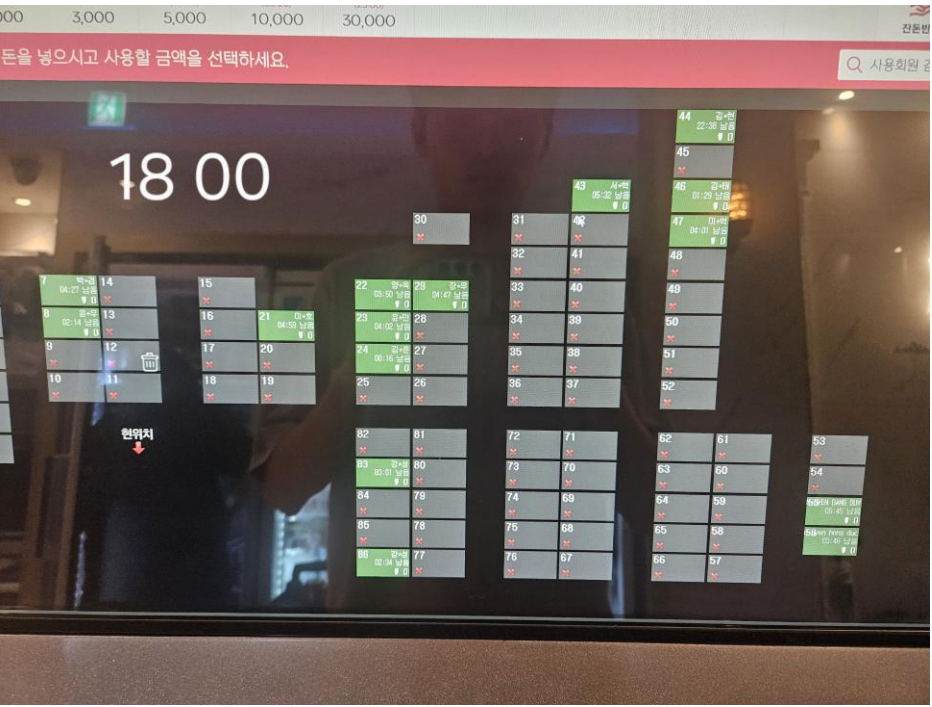
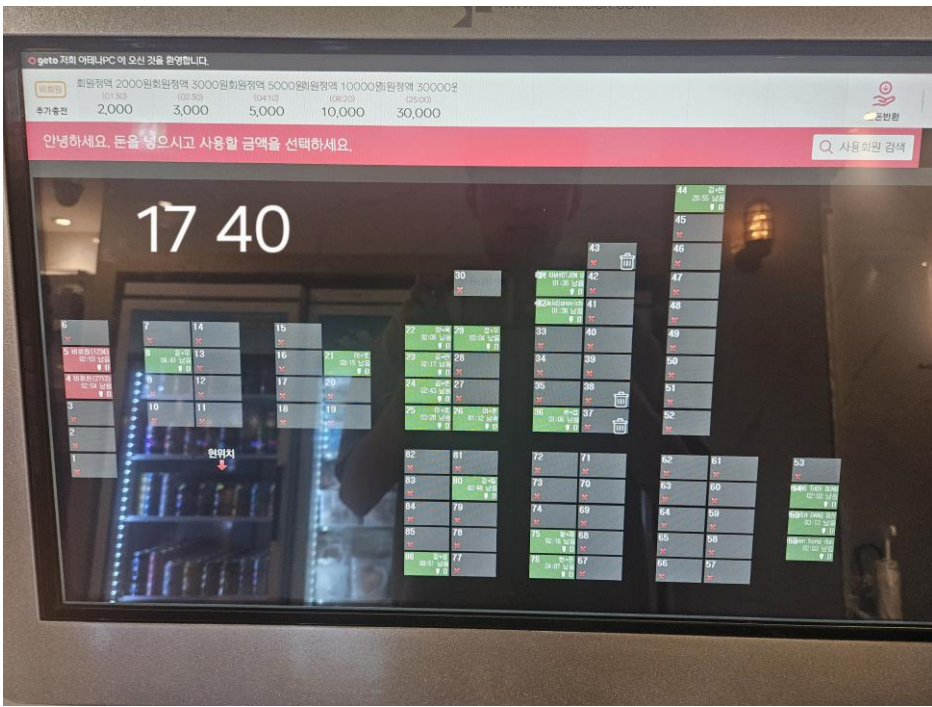
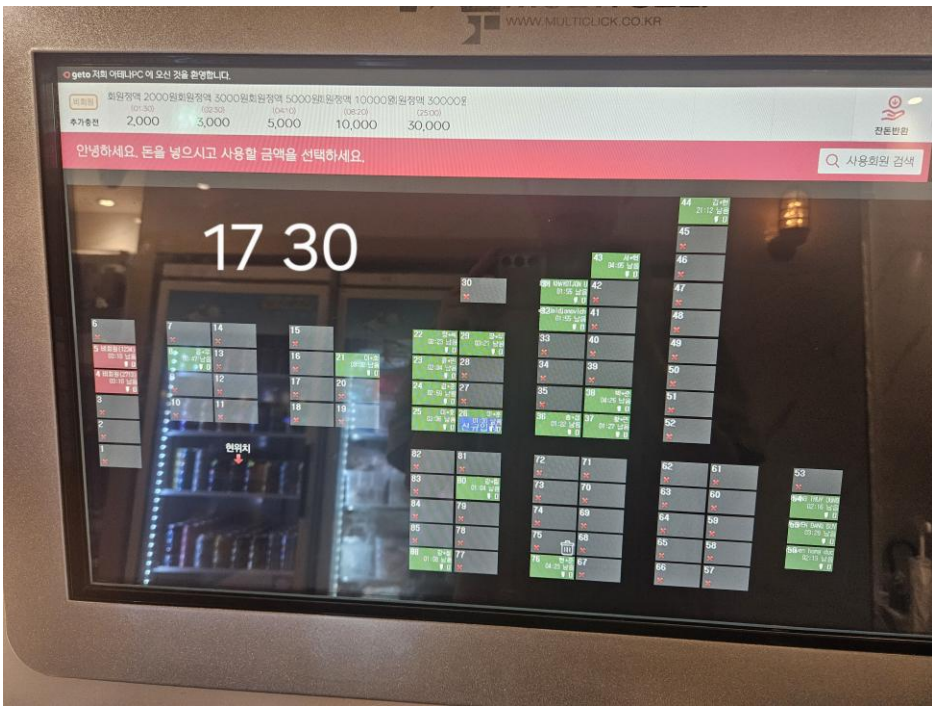
[1] Zúñiga, E. R., Moris, M. U., Syberfeldt, A., Fathi, M., & Rubio-Romero, J. C. (2020). A simulation-based optimization methodology for facility layout design in manufacturing. IEEE Access, 8, 163818-163828.

[2] 곽성일, & 이우진. (2018). PC 방 매출 극대화를 위한 그룹 고객 전용 연석 좌석 운영에 관한 연구. 대한산업공학회 추계학술대회 논문집, 2897-2911

02. 시뮬레이션 모델링 – Input Modeling

데이터 수집

설문조사	
1	PC방 방문 시 평균 체류 시간
2	PC방 이용 시 간식, 음료 등 추가 구매 여부
3	PC방 1회 방문 시 평균 추가 사용 금액
4	귀하만의 좌석 선택 기준 여부
5	선호하는 좌석 유형(1인석, 2인석 등)
6	단체로 방문했을 때, 자리가 없을 경우의 행동
현장조사	
1	단위 시간당 방문 손님 수
2	단체 손님의 유형별 방문 수
3	손님별 이용 시간



4:00~4:10	1 1 1 4
4:10~4:20	1 1 1 3
4:20~4:30	1 2 1 4 1 2 2 1 2
4:30~4:40	1 3 1 1 1
4:40~4:50	1 1 1
4:50~5:00	1 1
5:00~5:10	1 1
5:10~5:20	2 1 3 1
5:20~5:30	1 1
5:30~5:40	1
5:40~5:50	5
5:50~6:00	2 1 1

→ 실제 현장에서 조사한
단위 시간당 방문 손님 수 및 유형

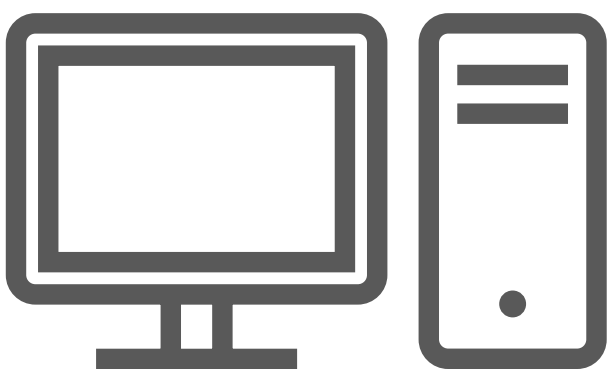
■ 시스템 가정 사항



<손님의 행동 원칙>

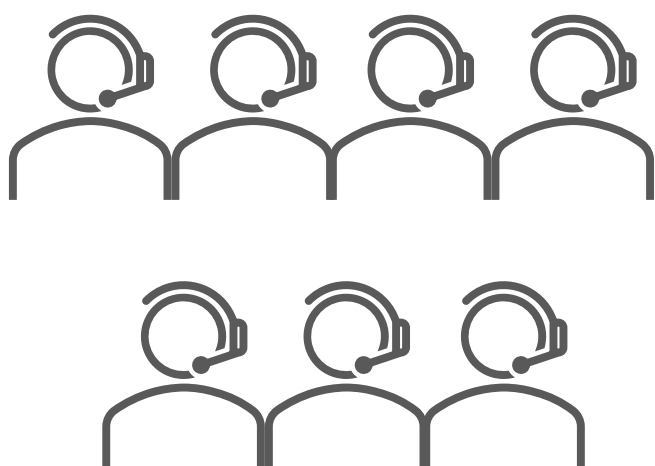
- 1. 손님은 랜덤하게 자리를 선택한다
- 2. 단체손님은 반드시 일렬로 연결된 좌석을 점유한다
 - 만약 좌석이 충분하지 않은 경우,
단체 손님(4-5명)은 22% 확률로 (2/2 or 2/3) 나눠 앉는다
나눠 앉을 의향이 없거나 개인 손님의 경우, 퇴장한다
- 3. 전체기간을 고려한다
 - 시간별(피크/비피크)로 수요 분포는 다름을 가정한다

*22%는 설문조사 기반 데이터



<PC방의 수익 구조>

- 1. 사용시간, 좌석 활용률에 비례
- 2. PC방 시간당 요금: 1200원
- 3. 단체 손님은 PC방의 가격이 낮을 경우,
기꺼이 나눠 앉을 비율이 늘어난다

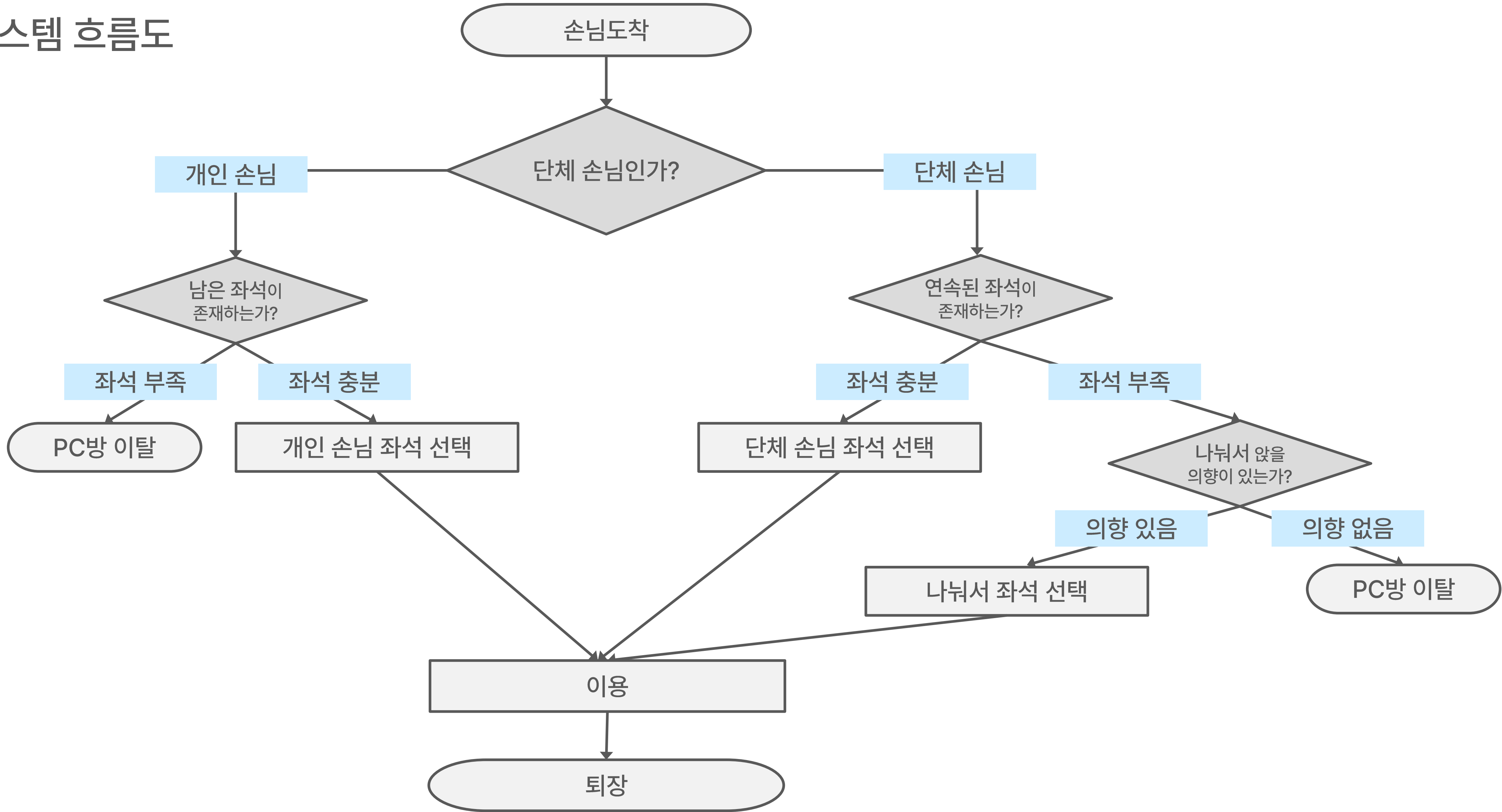


<단체손님 비율>

*설문조사 기반 데이터

단체손님 비율 (%)	
1	39.5
2	24.7
3	4.9
4	8.6
5	22.2

■ 시스템 흐름도



02. 시뮬레이션 모델링 - Input Modeling(도착 간격)

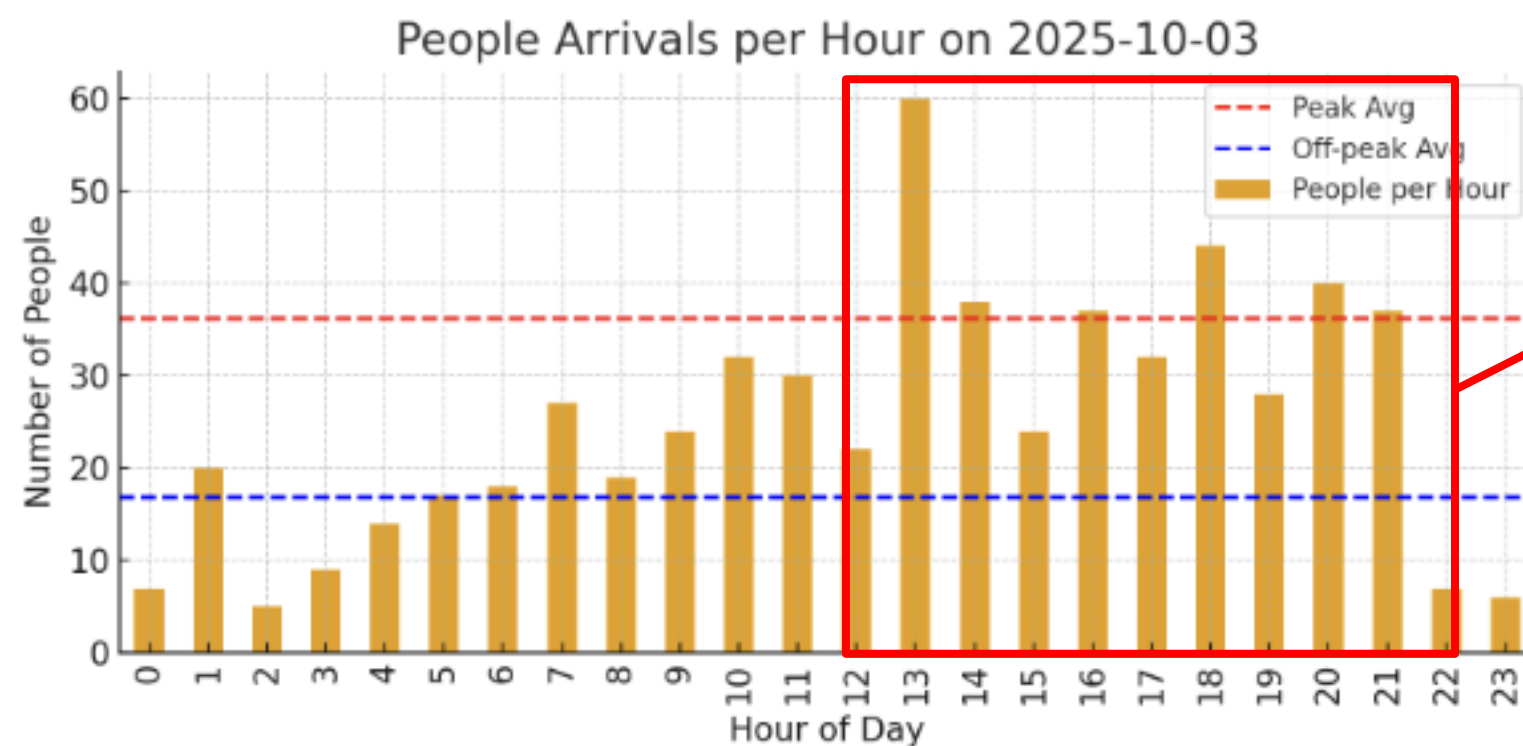
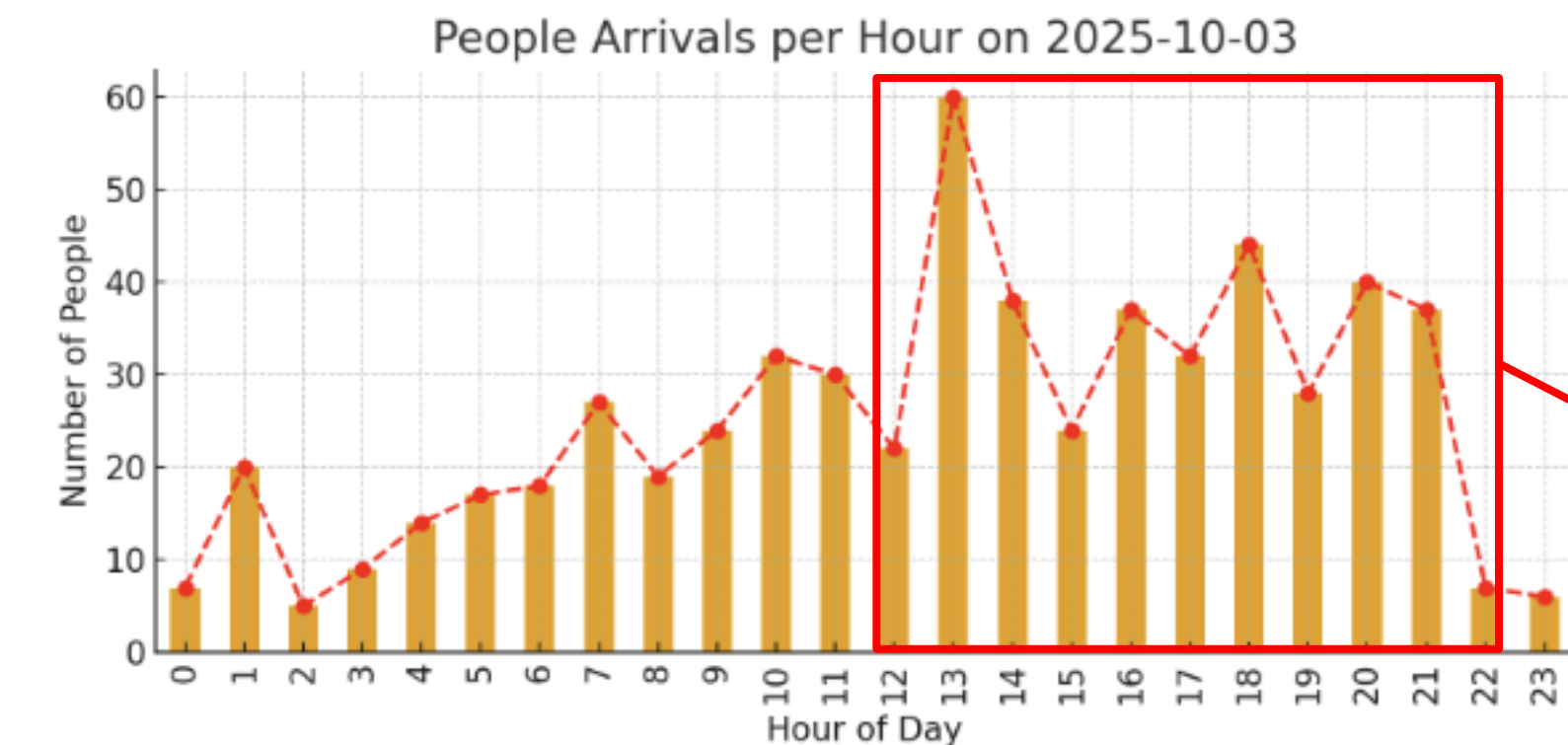
7

■ 피크/비피크 시간대 도착 분포 분석

시간대 분할 이유 : 고객 도착 분포는 **시간대별로 수요 차이**를 보임

→ 분석의 정확도를 높이기 위해 피크/비피크 시간대를 분리함

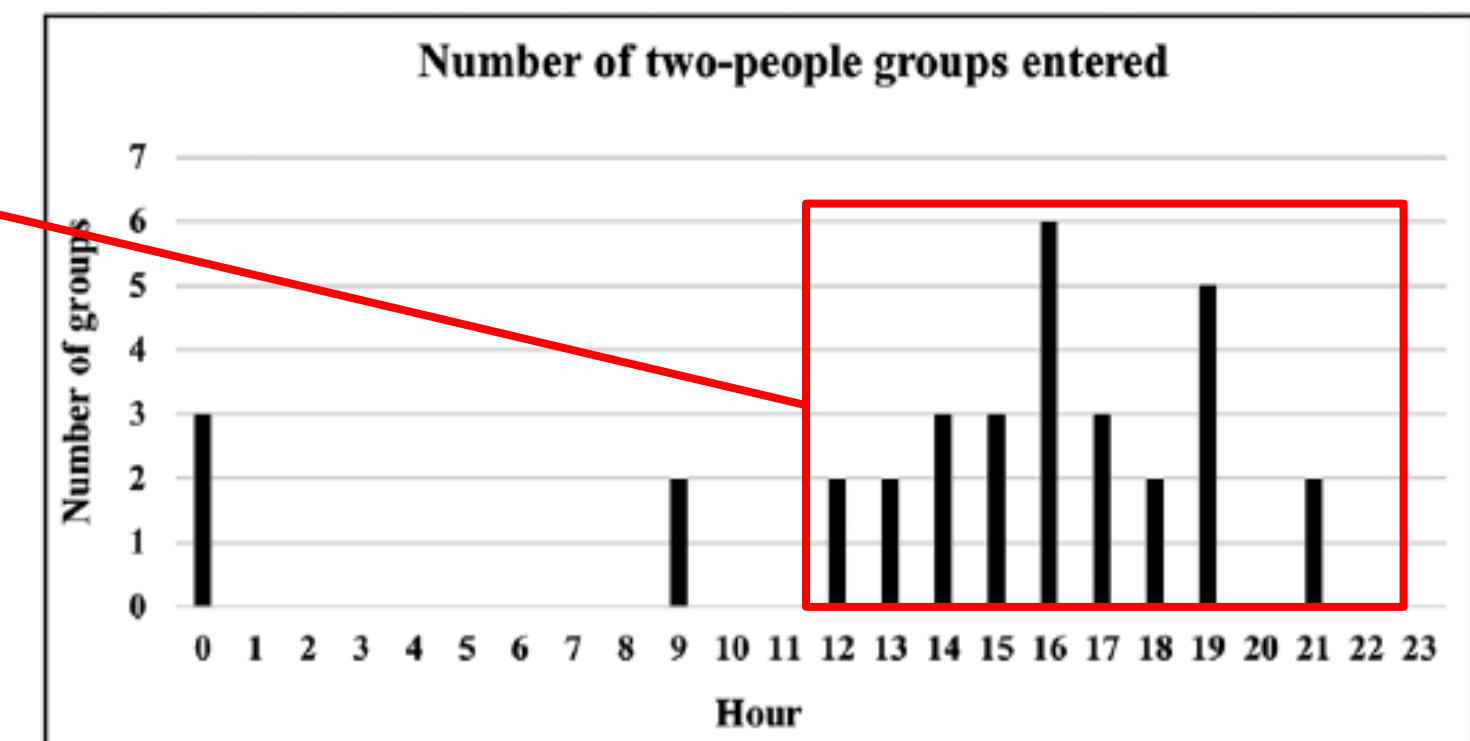
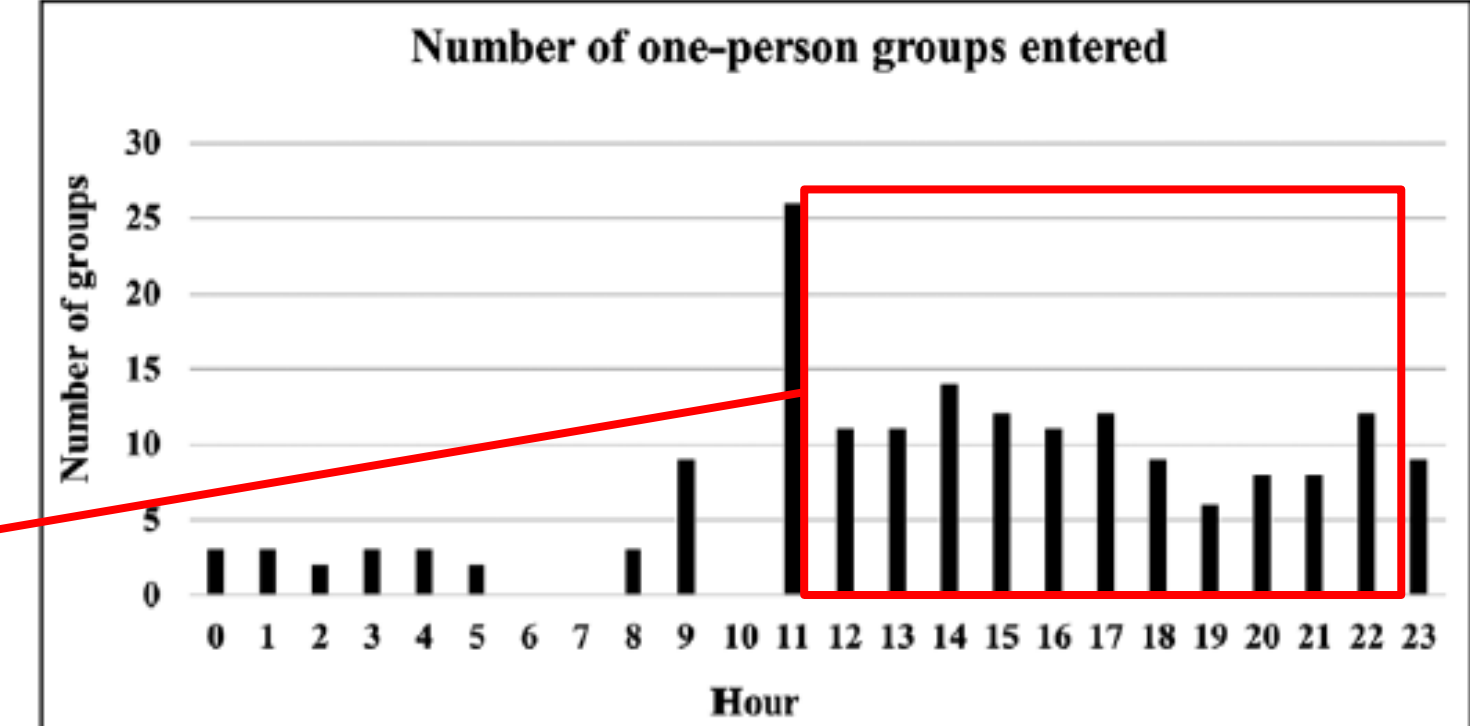
수집한 데이터 분포 확인



피크 시간대
→ 12:00 ~
22:00(10시간)

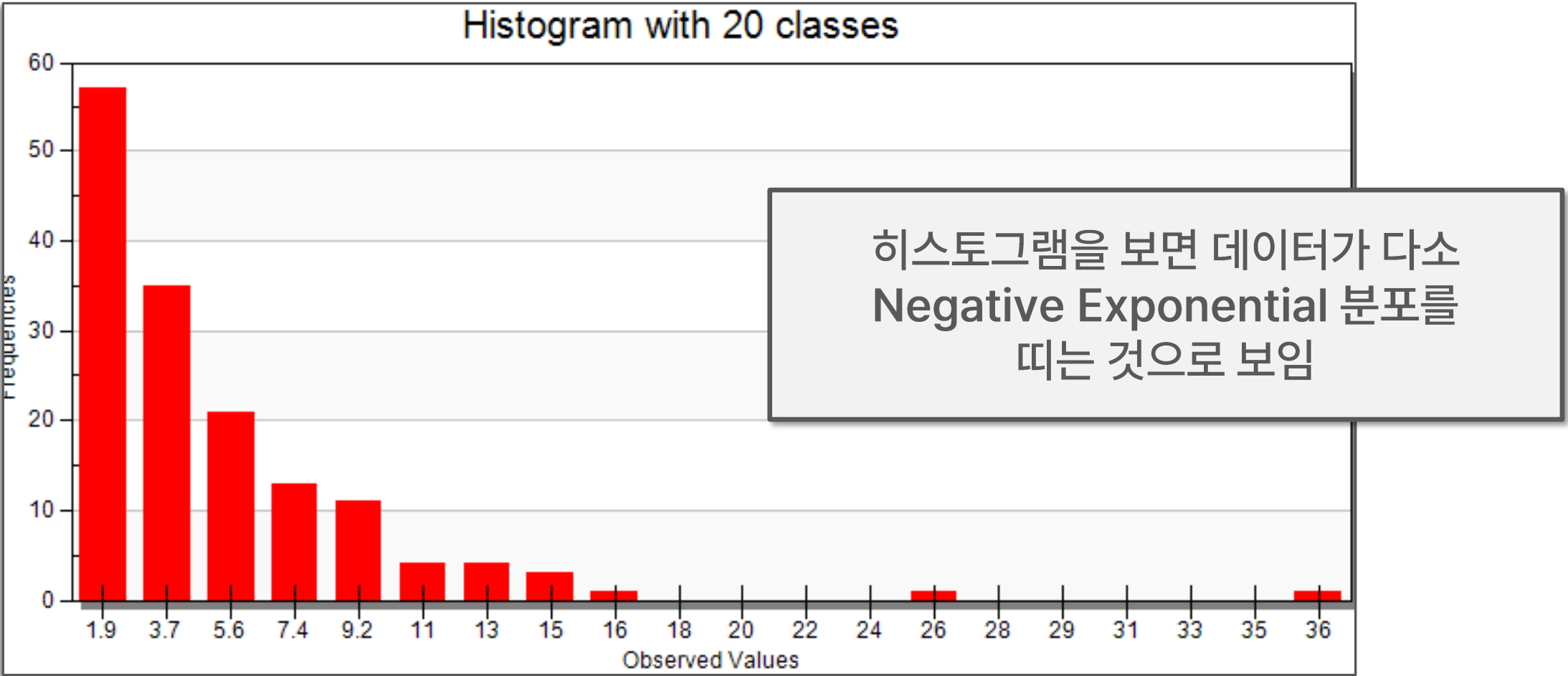
비피크 시간대
→ 22:00 ~
12:00(12시간)

참고 문헌 데이터 분포 확인



■ 피크 시간대(12:00~22:00, 10시간) 도착 간격 데이터 분석 및 결과

1. 히스토그램 분석



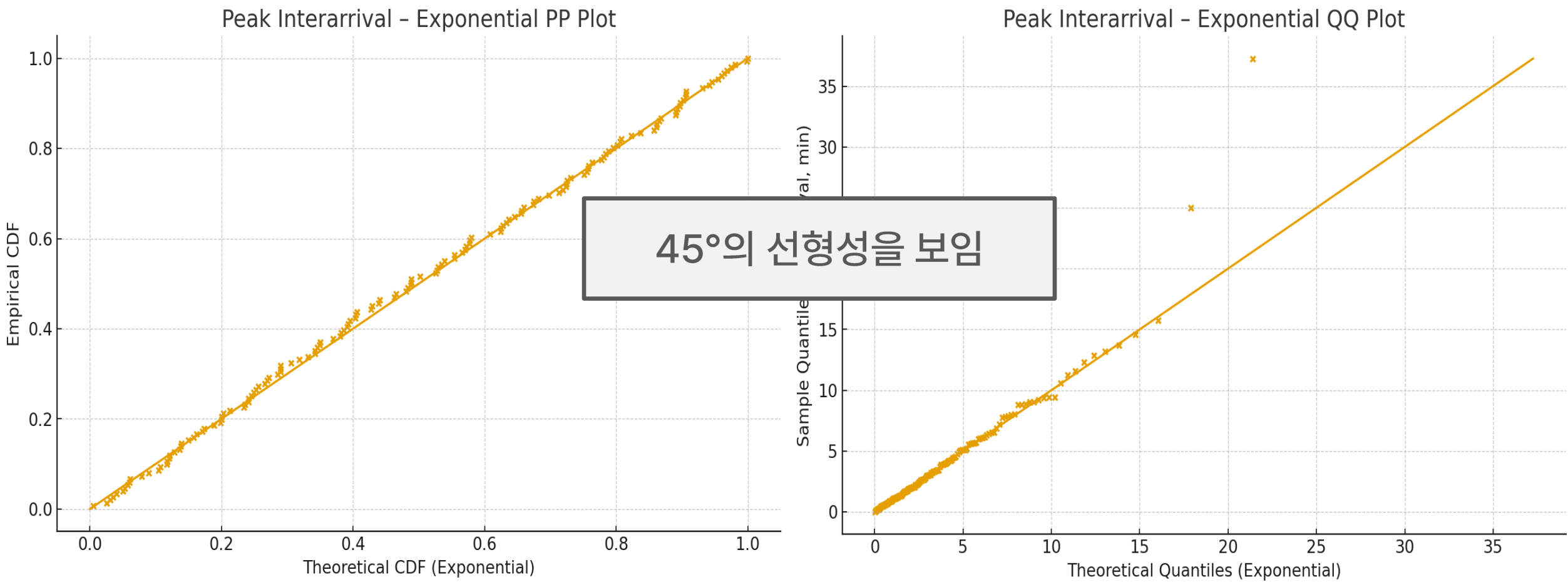
2. 적합도 검정

	string 0	real 1	real 2	boolean 3	real 4	real 5	boolean 6	real 7	real 8	boolean 9
string	Distributi...	Chi statistic	Chi value	Result Chi	KS statistic	KS value	Result KS	AD statistic	AD value	Result AD
1	Negexp	17.8742	28.8693	true	0.5117	1.3580	true	0.2328	2.4920	true
2	Erlang	13.3038	27.5871	true	0.5175	1.3580	true	0.2391	2.4920	true
3	Lognorm	14.1616	26.2962	true	0.8213	1.3580	true	1.0523	2.4920	true
4	Paralogistic	34.5629	27.5871	false	1.5713	1.3580	false	4.9354	2.4920	false
5	Gamma	10.9959	21.0261	true	1.8822	1.3580	false	1.5080	2.4920	true

⋮

Monte Carlo 시뮬레이션을 활용하여 분포 모수 추정을 2번 수행
→ 가정한 분포가 적합함을 확인!

3. Visual Evaluation



→ Sample Data가 가정한 확률분포에 대한 일치성이 높음

4. Input Modeling 결과

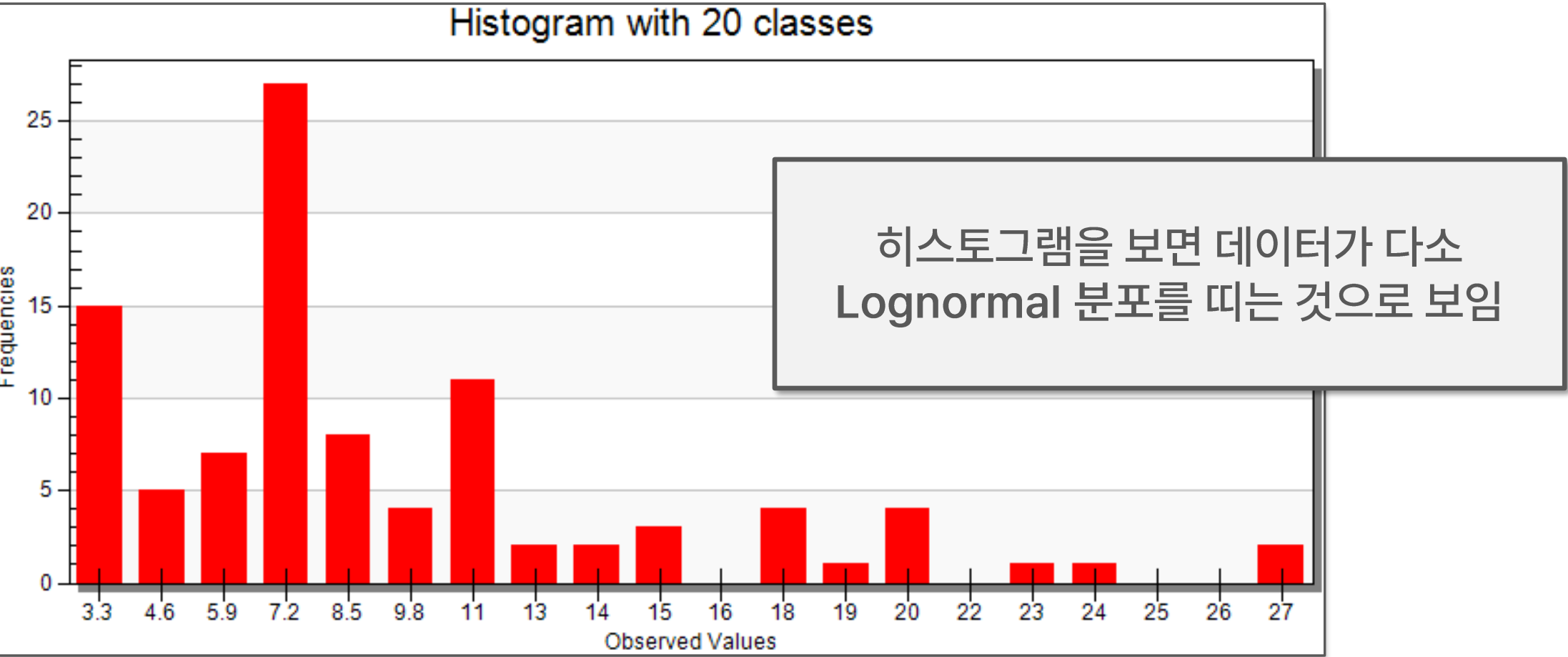
Distribution	Parameter1(λ)
Negexp	4.10434668874172

→ 위의 값을 모수로 가지는 Negative Exponential 분포를 따름

02. 시뮬레이션 모델링 - Input Modeling(도착 간격)

■ 비피크 시간대(22:00~12:00, 14시간) 도착 간격 데이터 분석 및 결과

1. 히스토그램 분석



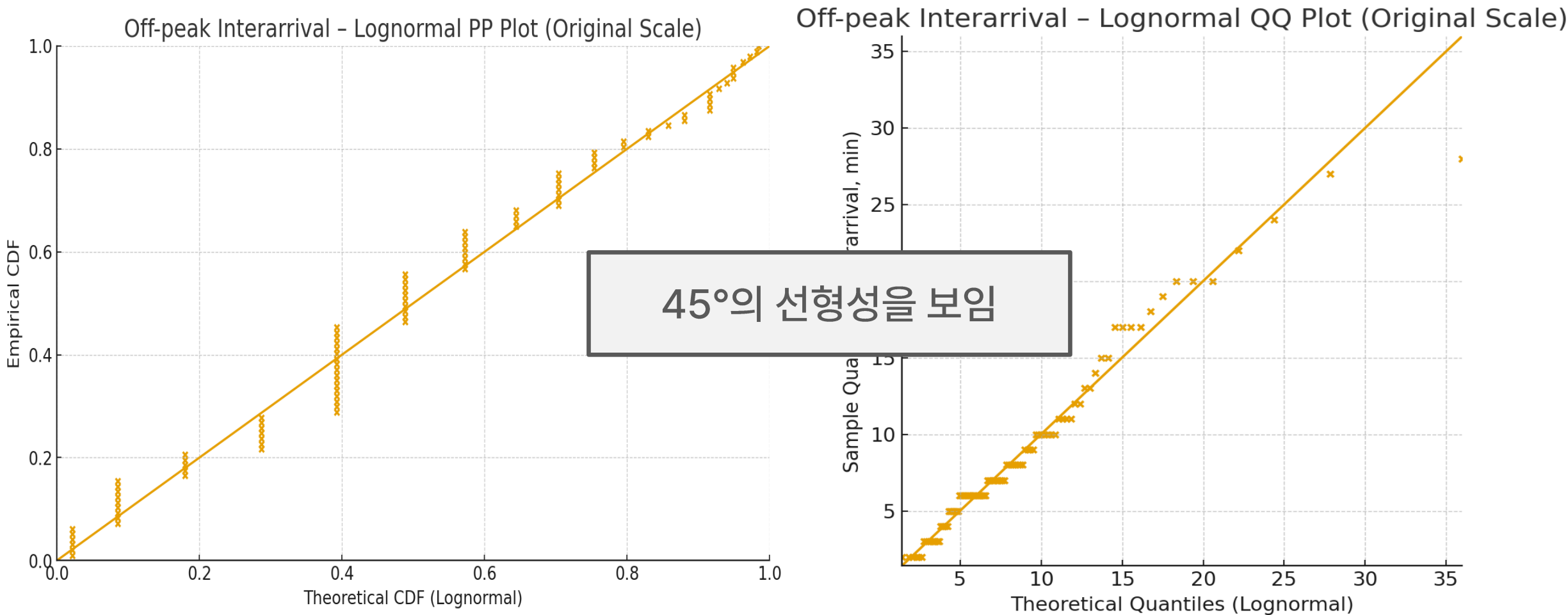
2. 적합도 검정

	string 0	real 1	real 2	boolean 3	real 4	real 5	boolean 6	real 7	real 8	boolean 9
string	Distributi...	Chi statistic	Chi value	Result Chi	KS statistic	KS value	Result KS	AD statistic	AD value	Result AD
1	Lognorm	26.2346	18.3070	false	0.7973	1.3580	true	0.6084	2.4920	true
2	Paralogistic	14.6495	14.0671	false	1.0669	1.3580	true	1.0014	2.4920	true
3	Gamma	40.2067	19.6751	false	1.0834	1.3580	true	0.9834	2.4920	true
4	Gumbel				1.0996	1.3580	true	1.0792	2.4920	true
5	Weibull	31.7629	14.0671	false	1.2902	1.3580	true	1.5122	2.4920	true

⋮

Monte Carlo 시뮬레이션을 활용하여 분포 모수 추정을 2번 수행
→ 가정한 분포가 적합함을 확인!

3. Visual Evaluation



→ Sample Data가 가정한 확률분포에 대한 일치성이 높음

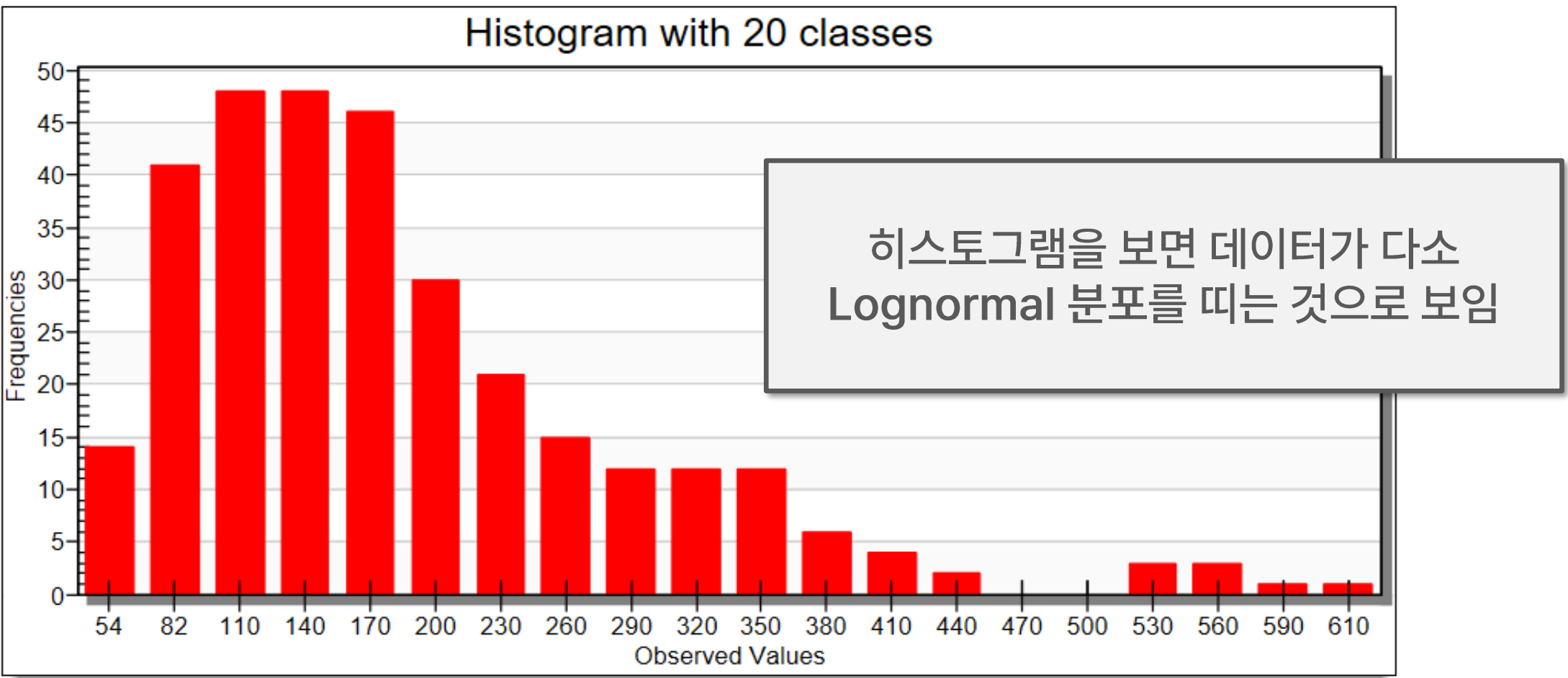
4. Input Modeling 결과

Distribution	Parameter1(μ)	Parameter2(σ)
Lognorm	8.88118401332568	5.39869957215745

→ 위의 값을 모수로 가지는 Lognormal 분포를 따름

■ 체류 시간 데이터 분석 및 결과

1. 히스토그램 분석



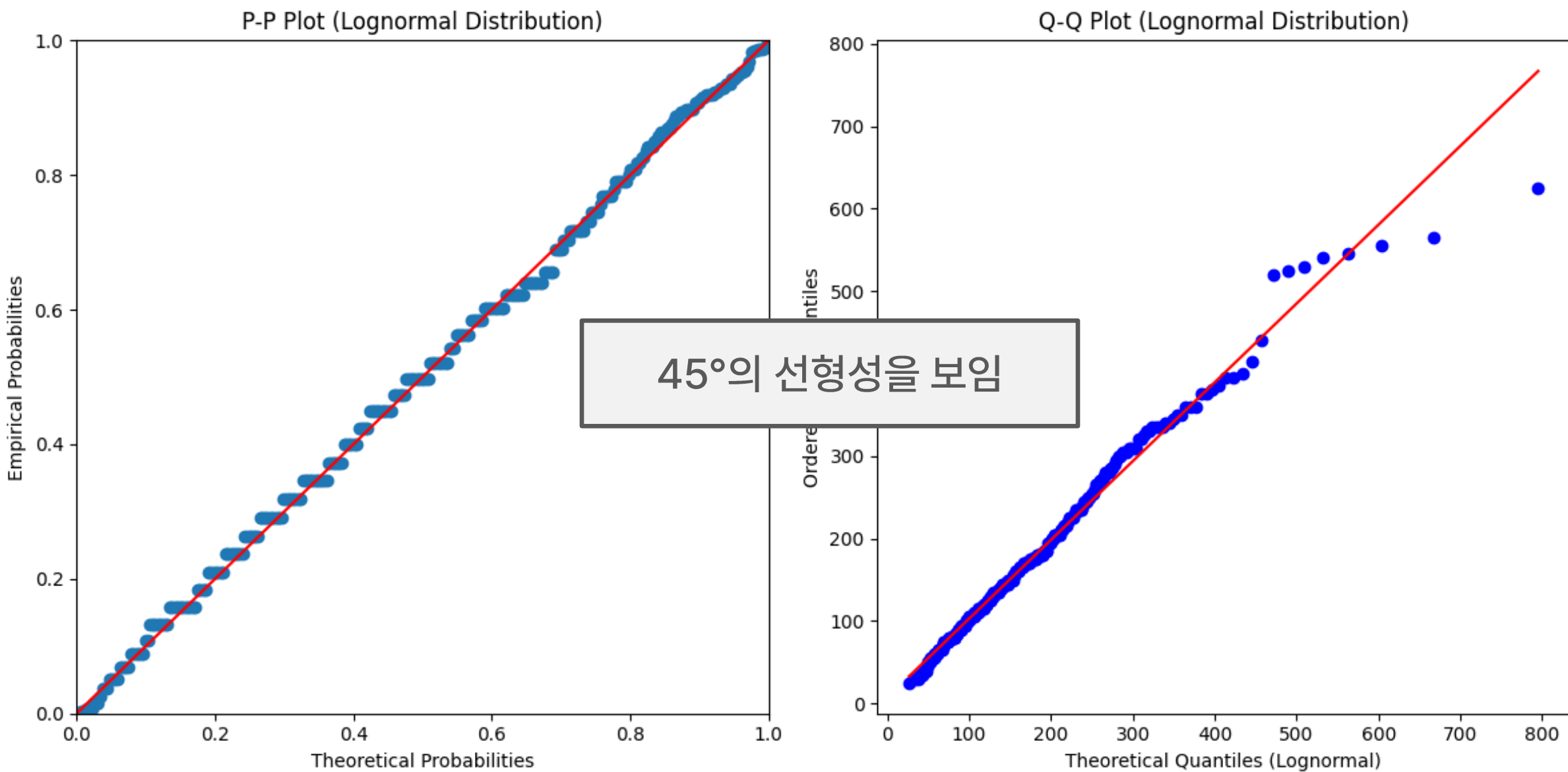
2. 적합도 검정

	string 0	real 1	real 2	boolean 3	real 4	real 5	boolean 6	real 7	real 8	boolean 9
string	Distribution	Chi statistic	Chi value	Result Chi	KS statistic	KS value	Result KS	AD statistic	AD value	Result AD
1	Lognorm	25.4463	27.5871	true	0.6343	1.3580	true	0.3387	2.4920	true
2	Paralogistic	29.7774	27.5871	false	1.0612	1.3580	true	1.0008	2.4920	true
3	Gamma	41.2895	27.5871	false	1.2793	1.3580	true	1.3348	2.4920	true
4	Erlang	41.5275	27.5871	false	1.0200	1.3580	true	1.1115	2.4920	true
5	Weibull	54.3542	27.5871	false	1.7171	1.3580	false	3.3953	2.4920	false

⋮

Statistical Tools의 DataFit 모듈을 사용하여
가정한 Lognormal 분포의 적합도 검정
→ 가정한 분포가 적합함을 확인!

3. Visual Evaluation



→ Sample Data가 가정한 확률분포에 대한 일치성이 높음

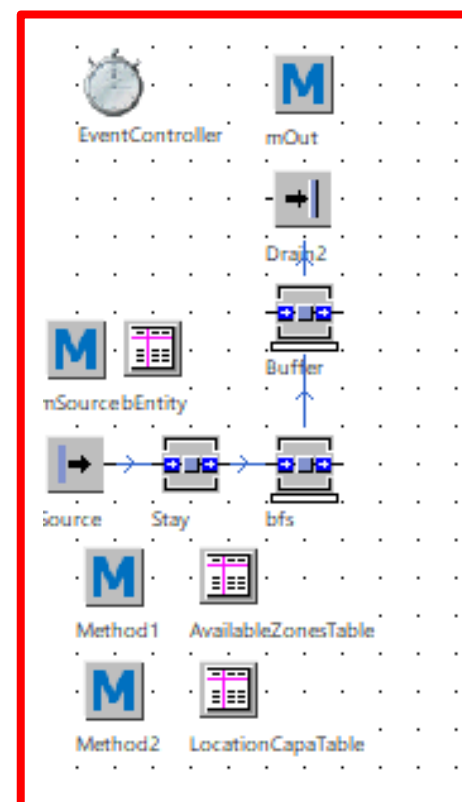
4. Input Modeling 결과

Distribution	Parameter1(μ)	Parameter2(σ)
Lognorm	173.979729700835	113.590204091523

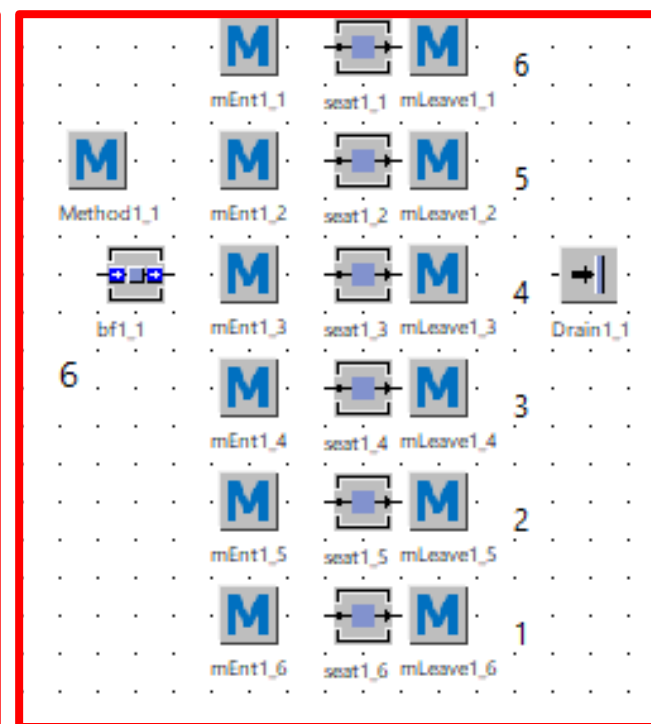
→ 위의 값을 모수로 가지는 Lognormal 분포를 따름

■ 전체 레이아웃 - 기존

①



②



← 나머지 좌석

■ 손님 Entity 정의

MyObjects

Customer1

Customer2

Customer3

Customer4

Customer5

Customer5_3

Customer5_2

Customer4_2

-기존 손님 Entity

.MyObjects.Customer1

Navigate View Tools Help

Name: Customer1

☐ Stopped

Label:

Conveying direction: 0 (forward)

Attributes

Routing

Graphics

Product Statistics

Costs

User-defined

+ New...

Edit...

Delete

Name	Value	Type	C..	I...
aJobStartTime	2022/01/01 00:00:00.0000	datetime	*	
aLeaveTime	2022/01/01 00:00:00.0000	datetime	*	
aType	1	integer	*	

OK

Cancel

Apply

-단체 손님이 나눠 앓는 경우의 손님 Entity

.MyObjects.Customer5_3

Navigate View Tools Help

Name: Customer5_3

☐ Stopped

Label:

Conveying direction: 0 (forward)

Attributes

Routing

Graphics

Product Statistics

Costs

User-defined

+ New...

Edit...

Delete

Name	Value	Type	C..	I...
aJobStartTime	2022/01/01 00:00:00.0000	datetime	*	
aLeaveTime	2022/01/01 00:00:00.0000	datetime	*	
aProcTime	0	real	*	
aType	3	integer		

OK

Cancel

Apply

- 손님 내부 속성 정의

속성명	설명	데이터 타입
aJobStartTime	작업 시작시각	Datetime
aLeaveTime	작업 종료시각	Datetime
aType	손님 수	Integer(1,2,3,4,5)
aProcTime	작업 실행 시간 저장 (분기 손님)	Real

좌석 Station 정의

The diagram illustrates a 6x6 grid of station icons. The icons are labeled as follows:

- Row 1: mEnt1_1, seat1_1, mLeave1_1, mEnt1_2, seat1_2, mLeave1_2, mEnt1_3, seat1_3, mLeave1_3, mEnt1_4, seat1_4, mLeave1_4, mEnt1_5, seat1_5, mLeave1_5, mEnt1_6, seat1_6, mLeave1_6
- Row 2: Method1_1, mEnt1_2, seat1_2, mLeave1_2, mEnt1_3, seat1_3, mLeave1_3, mEnt1_4, seat1_4, mLeave1_4, mEnt1_5, seat1_5, mLeave1_5, mEnt1_6, seat1_6, mLeave1_6
- Row 3: bf1_1, mEnt1_3, seat1_3, mLeave1_3, mEnt1_4, seat1_4, mLeave1_4, mEnt1_5, seat1_5, mLeave1_5, mEnt1_6, seat1_6, mLeave1_6
- Row 4: mEnt1_1, seat1_1, mLeave1_1, mEnt1_2, seat1_2, mLeave1_2, mEnt1_3, seat1_3, mLeave1_3, mEnt1_4, seat1_4, mLeave1_4, mEnt1_5, seat1_5, mLeave1_5, mEnt1_6, seat1_6, mLeave1_6
- Row 5: mEnt1_2, seat1_2, mLeave1_2, mEnt1_3, seat1_3, mLeave1_3, mEnt1_4, seat1_4, mLeave1_4, mEnt1_5, seat1_5, mLeave1_5, mEnt1_6, seat1_6, mLeave1_6
- Row 6: mEnt1_3, seat1_3, mLeave1_3, mEnt1_4, seat1_4, mLeave1_4, mEnt1_5, seat1_5, mLeave1_5, mEnt1_6, seat1_6, mLeave1_6

Two screenshots of the station configuration windows are shown:

bf1_1 Configuration Window:

- Name: bf1_1
- Label:
- Planned
- MaxCapa: 6 (integer)
- MaxCapaM: (Double-click to open) (method)

seat1_1 Configuration Window:

- Name: seat1_1
- Label:
- Planned
- aCapa: 6 (integer)
- aCapaM: (Double-click to open) (method)
- aEnter: 1 (integer)

- 좌석 묶음 입구 버퍼 속성 정의

속성명	설명	데이터 타입
MaxCapa	현재 좌석 묶음 잔여 용량 표기	Integer
MaxCapaM	잔여 용량 계산기	Method

- 좌석 속성 정의

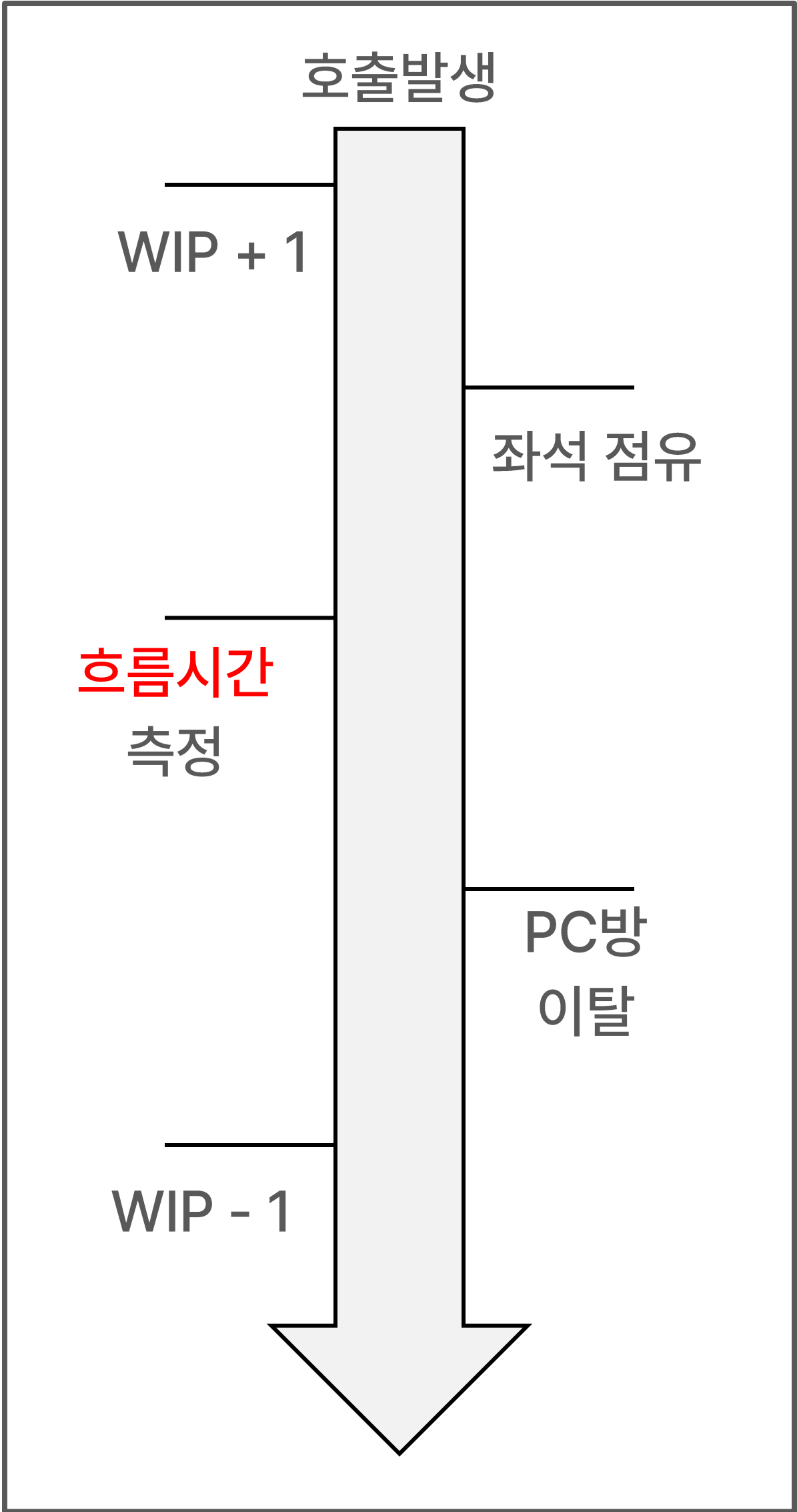
속성명	설명	데이터 타입
aCapa	현재 좌석에 진입 가능한 최대 손님 수	Integer
aCapaM	최대 손님 계산기	Method
aEnter	진입 가능 여부 (가능하다면 1, 아니면 0)	Integer(0, 1)

-생산성 지표 및 시뮬레이션 진행 파악을 위한 변수 설정

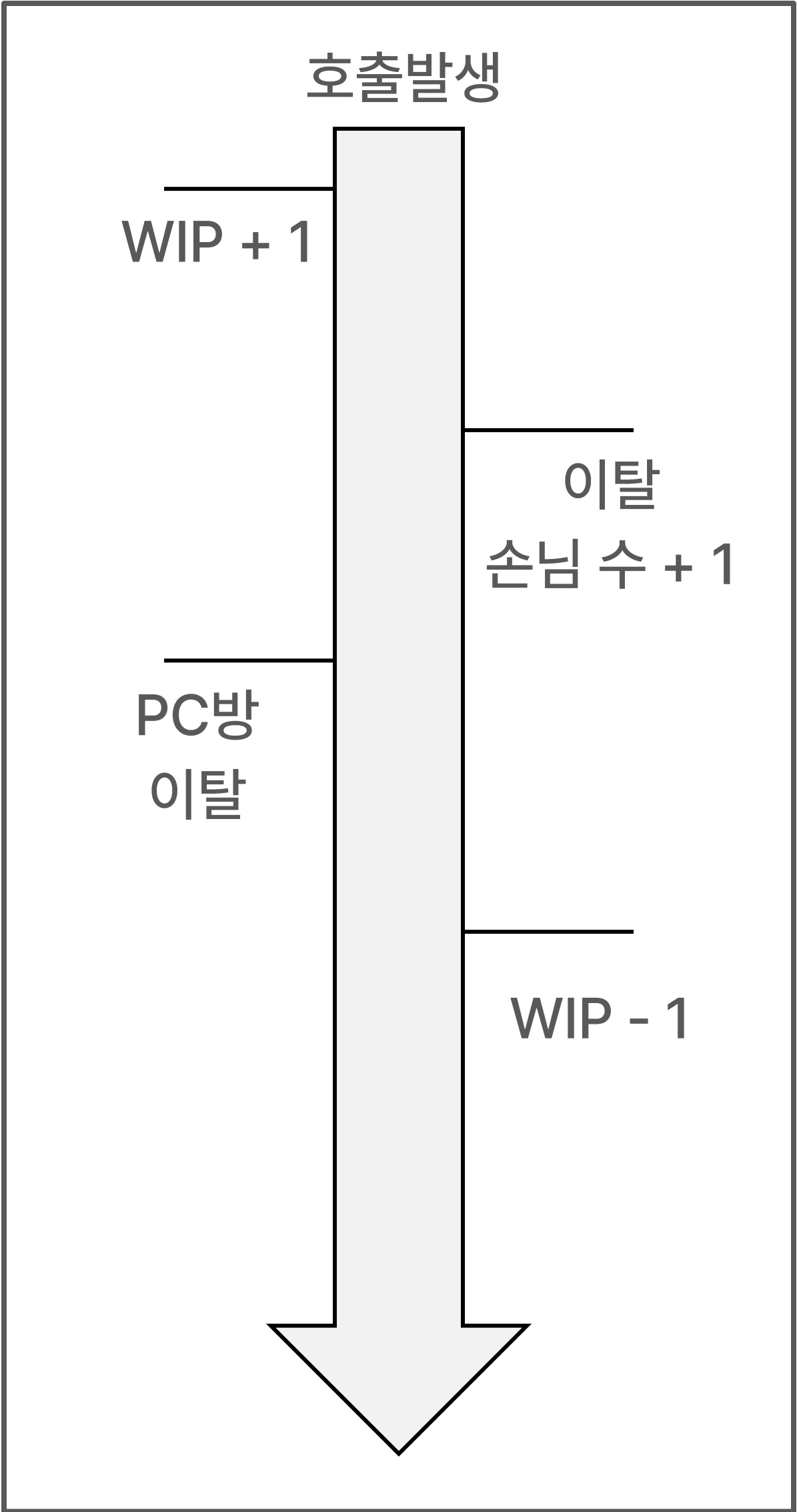
변수명	설명
Variable1	서비스시간 평균
Variable2	서비스시간 표준편차
vDividedPer	나눠질 의향이 있는 손님 비율
vTotal_Served_Customer	시스템에 들어온 총 단체 수
vTotal_notServed	시스템에서 서비스를 받지않고 이탈한 총 단체 수
vExit_Customer_per	이탈률(단체)
vNumSeats	좌석 수(85)
vEntireTime	시뮬레이션 기간(30일)
vSeatAvail	평균 좌석 활용률
vCostperTime	시간당 PC방 이용 요금
vTotalCost	PC방 총 수익
vWIP	PC방 운영 중 평균 손님 수
vFT	평균 손님 체류 시간
vTH	평균 흐름율

: 주요 KPI

<서비스를 받는 손님>

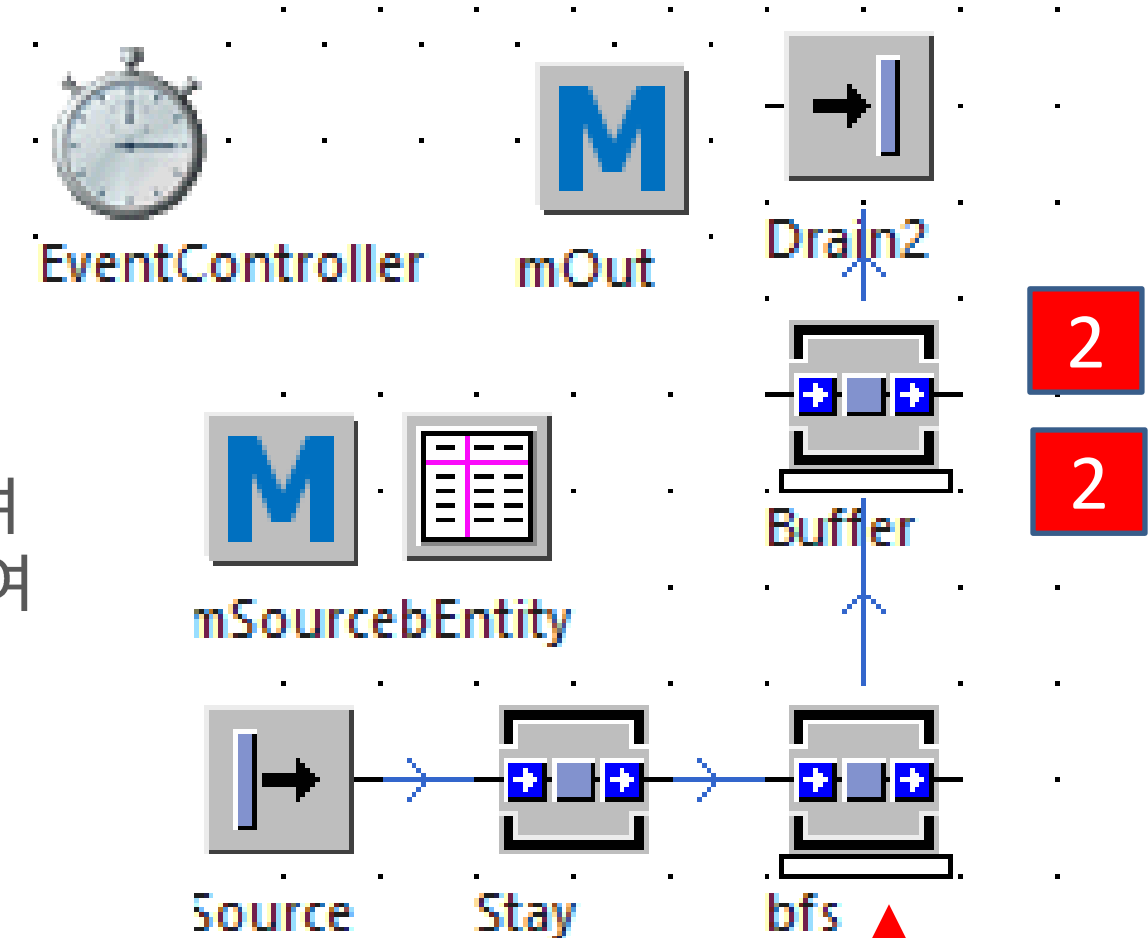


<이탈 손님>



<전체 시뮬레이션 모델링(예시)>

- 1. 나눠질 의향이 있고, 2자리가 비는 구역이 2개 이상인 경우 분할 및 기존 엔티티 제거
- 2. 분할된 경우 같은 서비스 시간 부여 및 bfs로 이동
- 3. 분할 안된 경우, Drain2로 이동



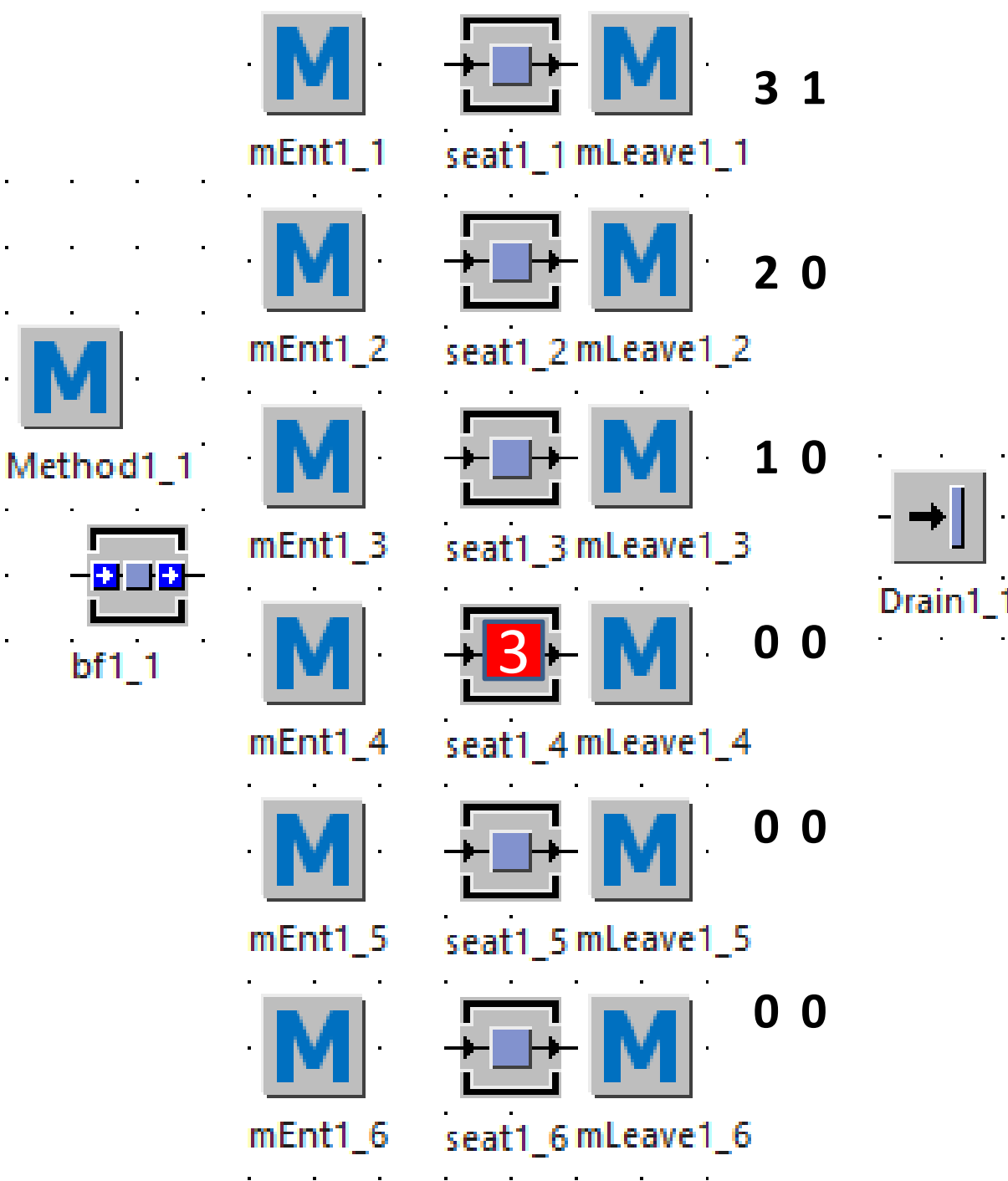
- 1. 나뉠 여부가 있으면 aDivide 부여
- 2. 피크시간에 따라서 도착 간격 부여

4

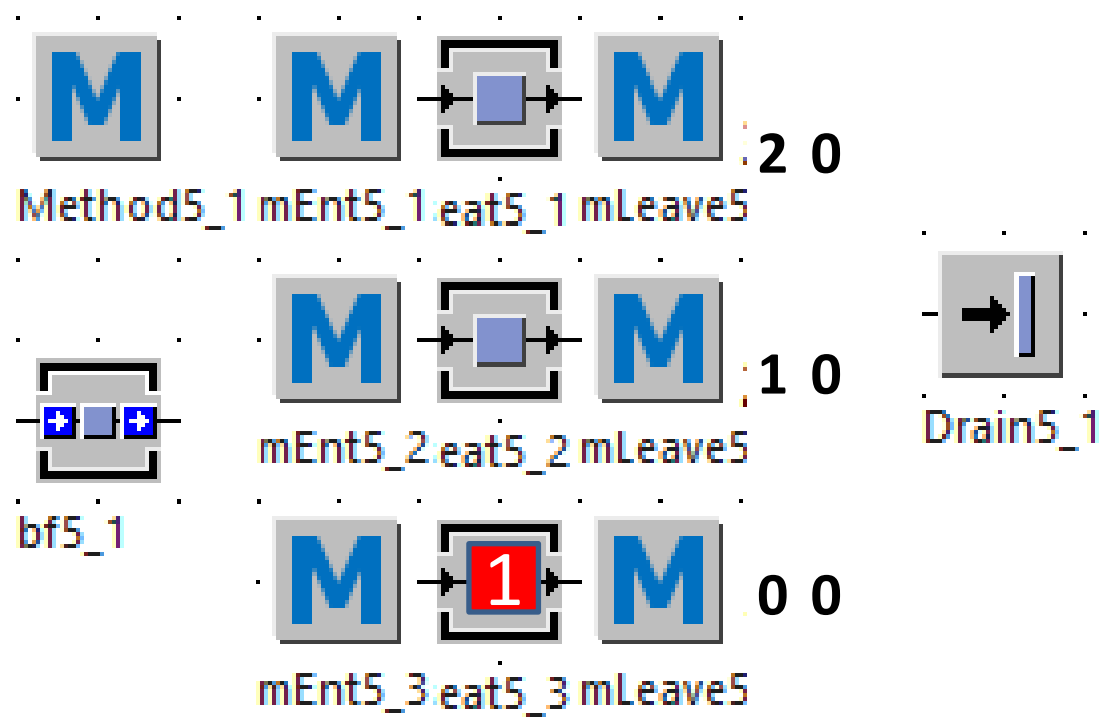
	object 1	integer 2
1	bf1_1	5
2	bf2_1	5
3	bf3_1	5
4	bf4_1	5
5	bf5_1	5
6	bf7_1	4
7	bf8_1	2
8	bf9_1	2
9	bf11_2	5
10	bf6_2	5
11	bf7_2	5
12	bf8_2	5
13	bf9_2	5
14	bf10_2	5
15	bf12_2	5
16	bf13_2	5
17	bf11	1
18	bf22	1

1. 손님이 갈수 있는 좌석 묶음 입구 버퍼 확인 후 AvailableZonesTable에 추가함

2. AvailableZonesTable에 가능 구역 존재 시, 해당 구역으로 이동. 없으면 Buffer로 이동

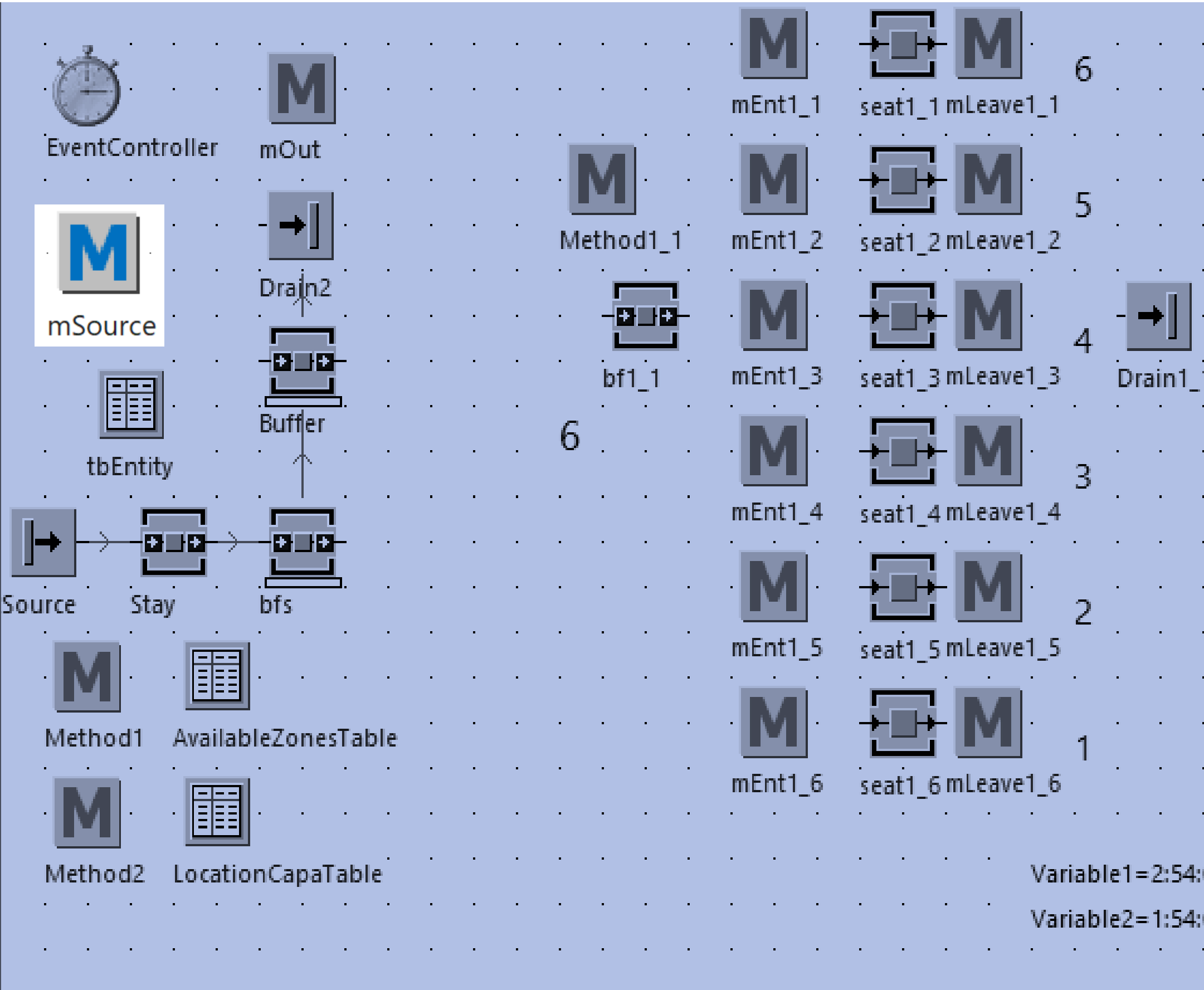


- 1. 출입 가능하고, 수용 가능 인원이 손님 수보다 많거나 같으면 할당
- 2. 할당이 된 경우 대표 좌석 사용불가 처리, 단체손님(2~5인) 경우 종속좌석 사용불가 처리



- 1. 사용을 마치고 좌석을 떠날 때, 좌석을 사용 가능 처리. 단체손님인 경우, 종속 좌석도 사용 가능 처리
- 2. 손님 퇴장 (Drain)

■ 코드 검증 과정



1. Source에서 생성하는 고객의 분할할 의향(aDivide)이 False에서 True 로 잘 바뀌는 지 검증

Four
.MyObjects.Customer4:961
on Source
Next event: Out (+0.0000)
FinishedTime: 30:00:12:02.0792

Name	Value	Type	C..
aBad	0	integer	
aDivide	true	boolean	
aJobStartTime	2022/01/01 00:00:00.0000	datetime	
aLeaveTime	2022/01/01 00:00:00.0000	datetime	
aSmoke	false	boolean	

고객이 분할할 확률을 1로 설정하고 기본값을 False로 설정 시 위와 같이 Source에서 생성되는 entity들의 분할할 확률이 True로 바뀌는 것 확인가능

<시물레이션 Entity 생성기 및 좌석의 일부>

■ 코드 검증 과정

```
PeakStart_Sec := 12 * 3600 -- 12시 00분 00초 = 43200초
PeakEnd_Sec   := 16 * 3600 -- 16시 00분 00초 = 57600초
```

```
if (CurrentTime_Sec >= PeakStart_Sec) and (CurrentTime_Sec < PeakEnd_Sec) then
    NextArrivalTime := z_negexp(1, 200)
else
    NextArrivalTime := z_negexp(1, 450)
end
```

<현재 가정>

피크시간 : 12시~16시
비피크시간 : 이 외의 시간

피크시간 고객 도착간격 : 200초
비피크시간 고객 도착 간격 :450초

<mSource의 피크시간대 설정과 피크시간 비피크시간 고객 도착간격 로직>

<실제 검증>

vPeak=59

vNonPeak=169

1-2. 평균 도착 간격

$$\text{평균 도착 간격} = \frac{\text{총 시간}}{\text{고객 수}} = \frac{14,400\text{초}}{59\text{명}} \approx 244.07\text{초/명}$$

• 피크 시간대 평균 도착 간격: 약 244.07초 (4분 4.07초)

2-2. 평균 도착 간격

$$\text{평균 도착 간격} = \frac{\text{총 시간}}{\text{고객 수}} = \frac{72,000\text{초}}{169\text{명}} \approx 426.04\text{초/명}$$

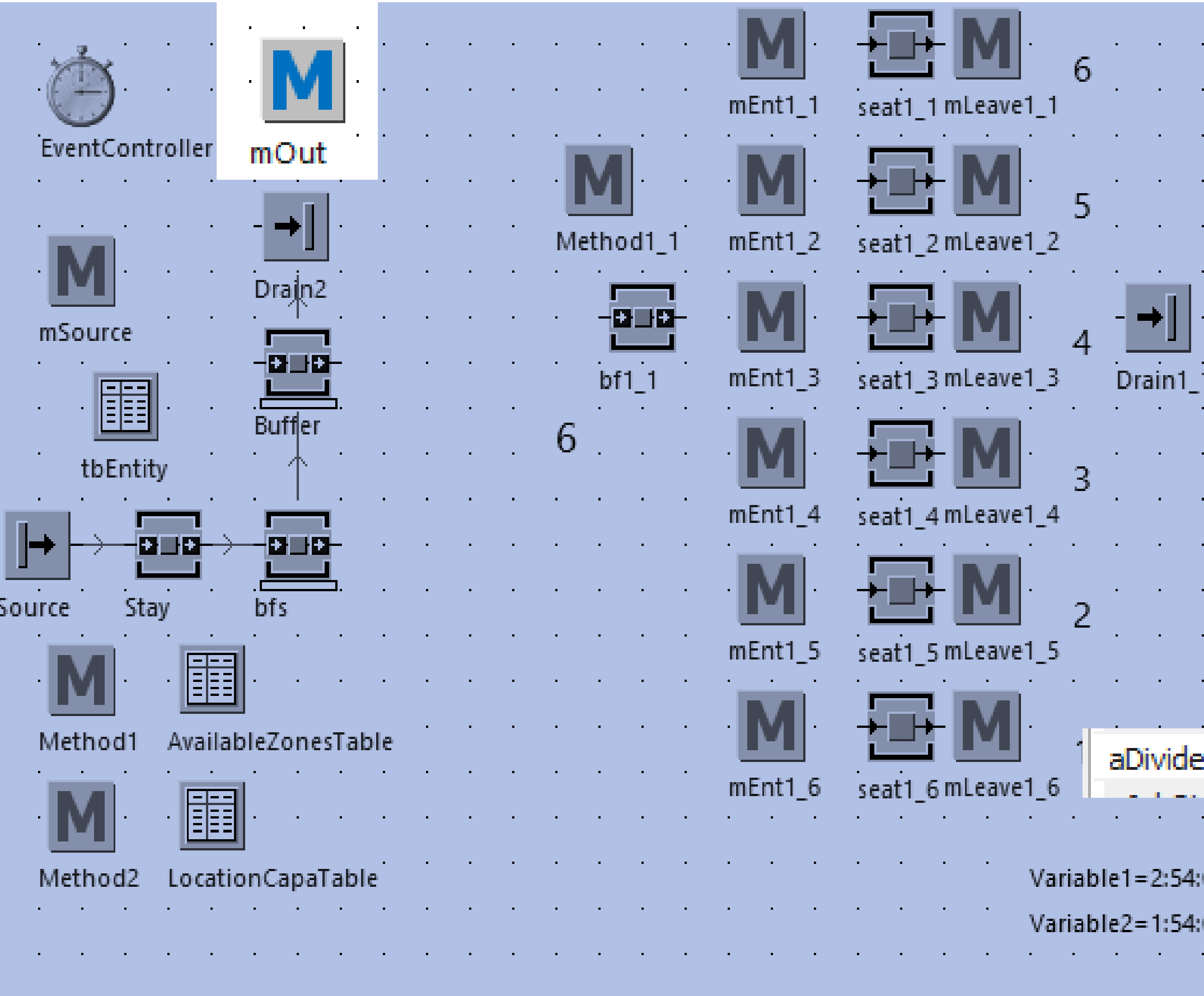
• 피크 아닌 시간대 평균 도착 간격: 약 426.04초 (7분 6.04초)

변동성에 의한 차이가 있을 수 있지만

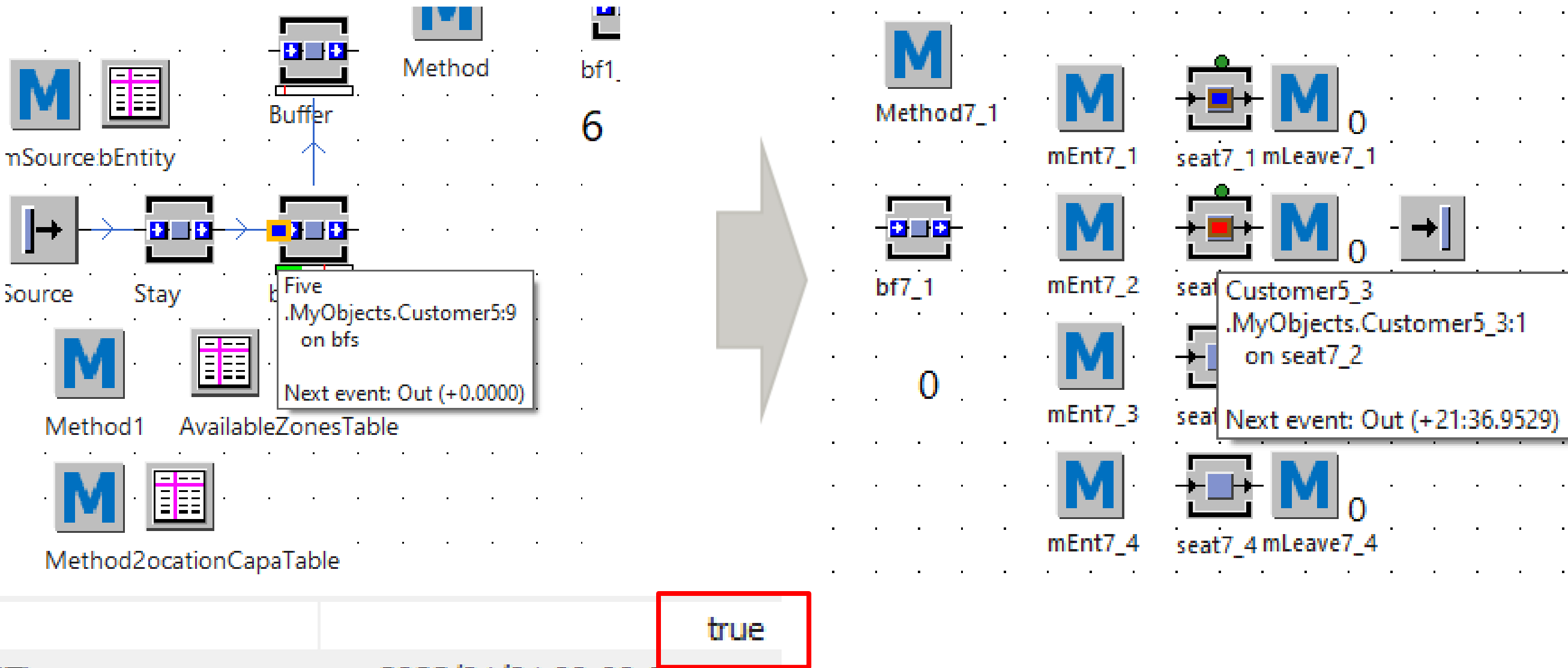
가정 피크시간대 도착간격 200초 ≈ 244초(실제)
가정 비피크시간대 도착간격 450초 ≈ 426초(실제)

<피크시간, 비피크시간 도착고객 카운팅 변수>

■ 코드 검증 과정



1. 나눠질 의향이 있는 고객 엔티티를 두 엔티티로 잘 나누는지 검증



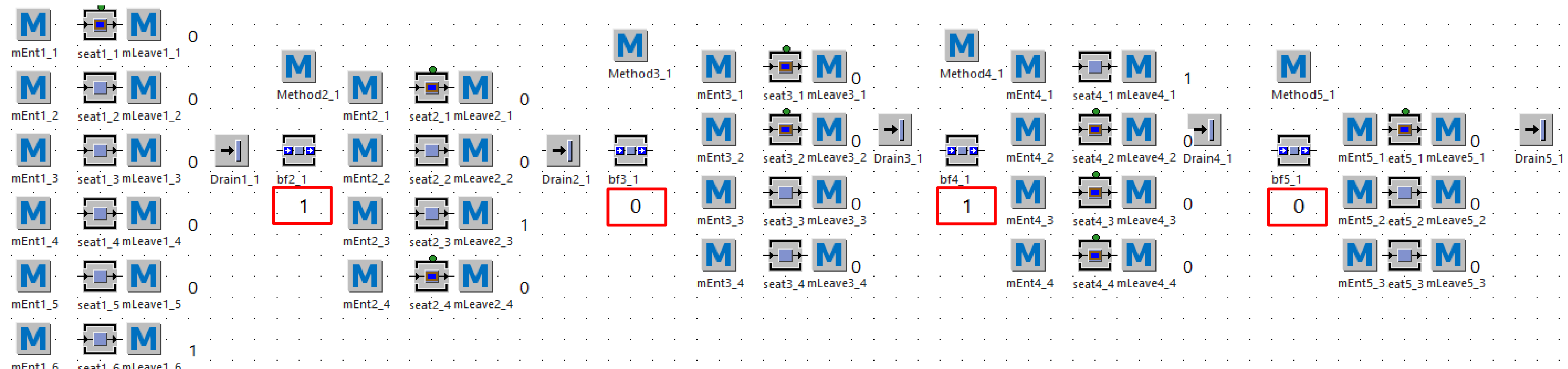
나눠질 의향이 있는 5인 단체 고객을 2명, 3명으로 나눈 후 원래 생성된 5인 단체 고객을 폐기 처리한 것을 볼 수 있음

<시물레이션 Entity 생성기 및 좌석의 일부>

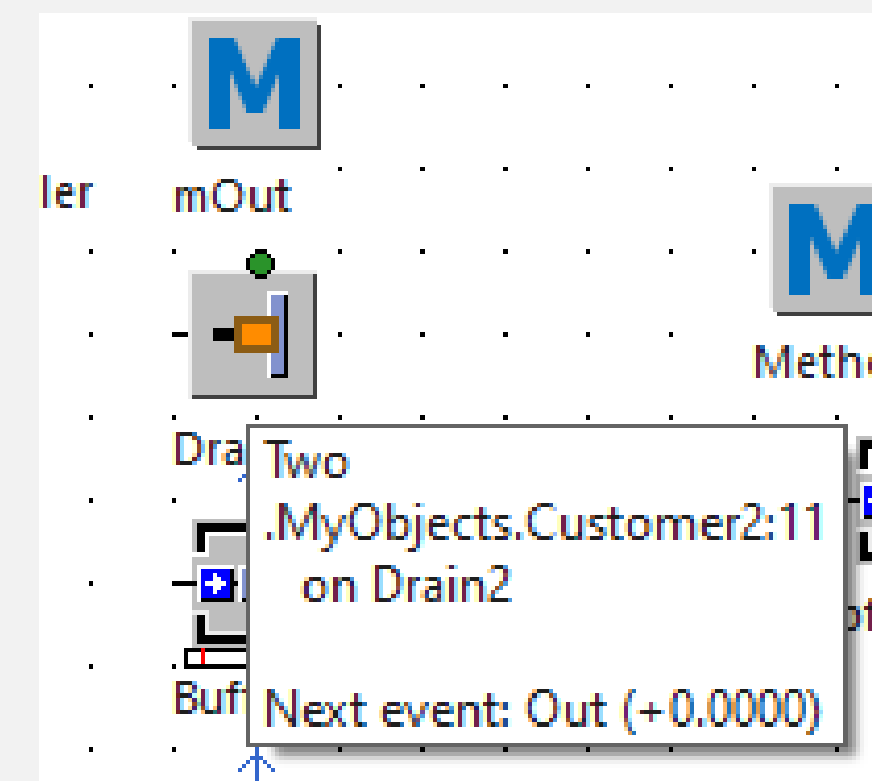
03-1. Verification – mOut

■ 코드 검증 과정

2. 자리가 꽉 차있을 시 drain2(폐기)로 보내는지 확인

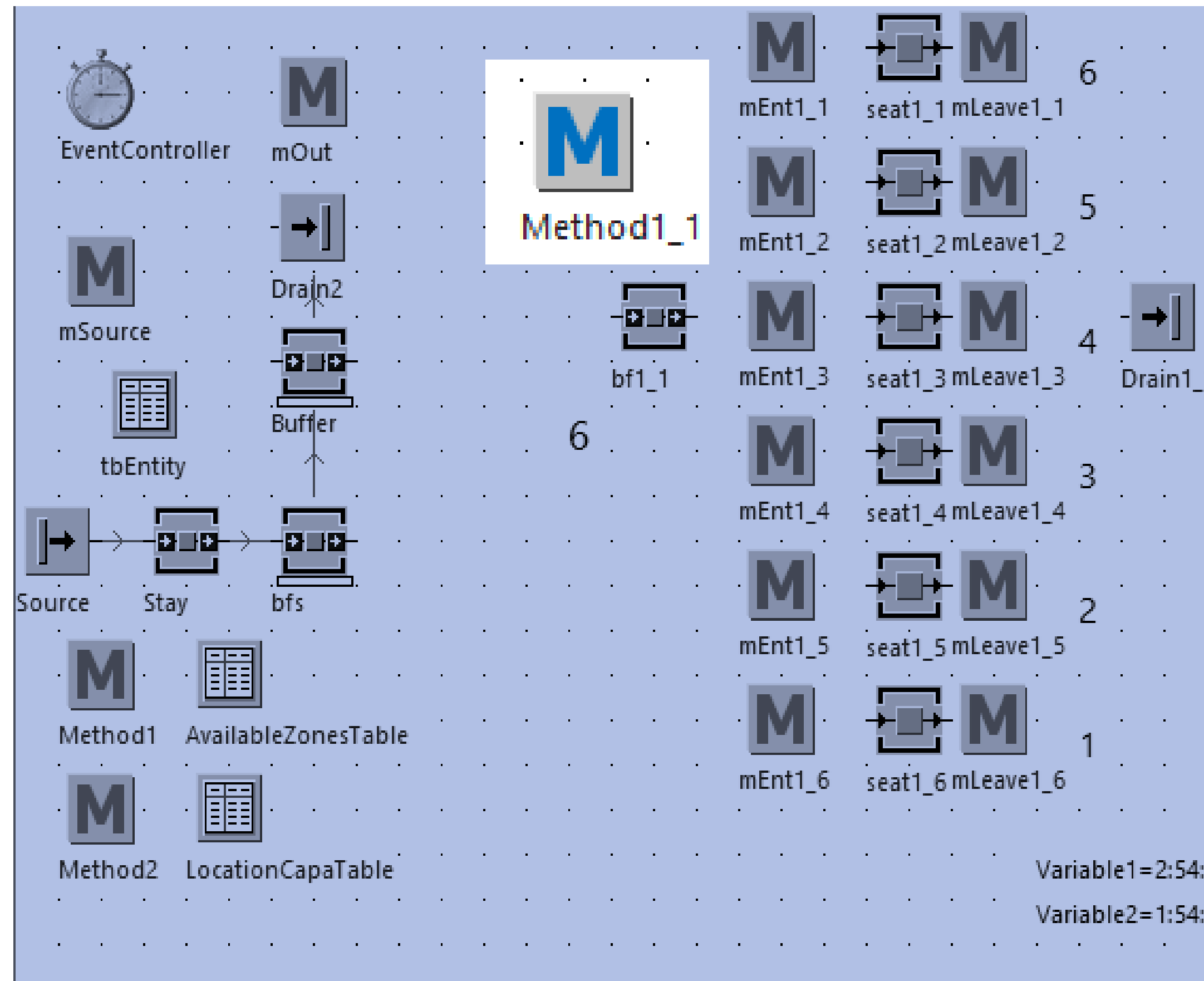


1 : 현재 자리 중 남은 자리의 개수



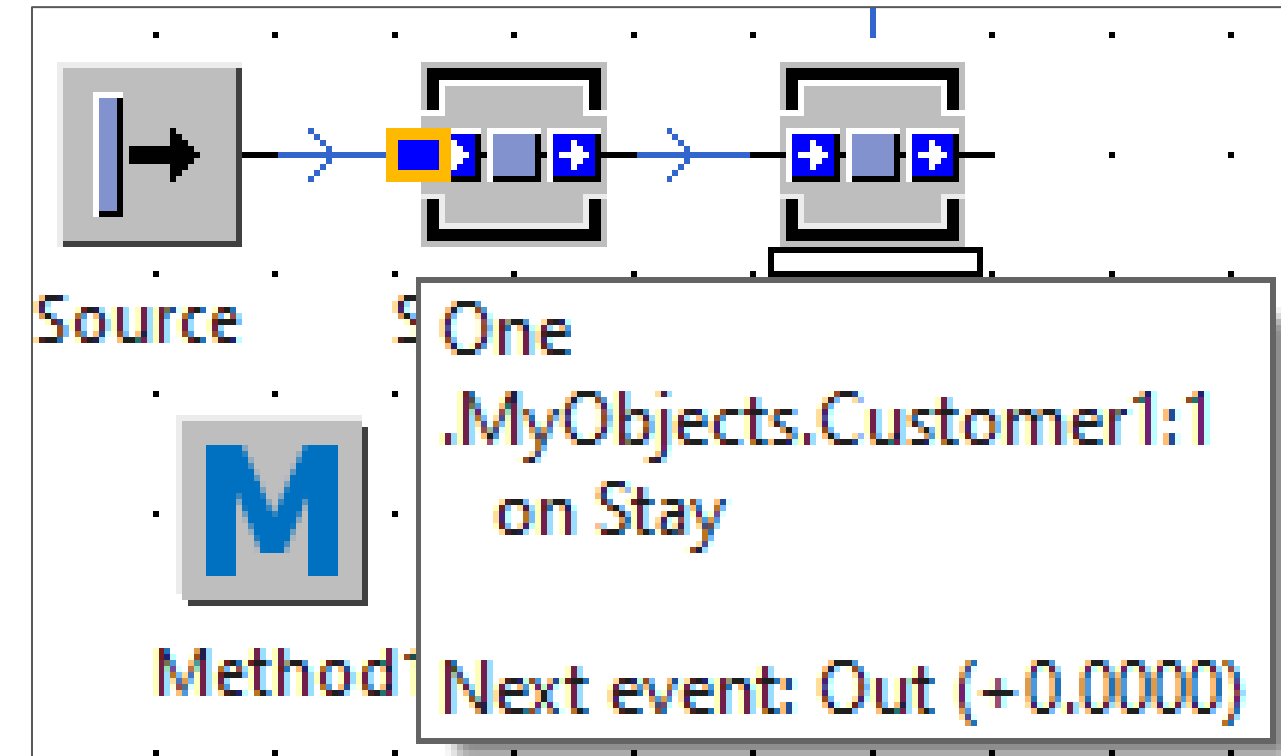
남은 자리가 1자리 이하로
남은 상황에서 두명의 손님이
오니 Drain2(폐기)로
잘 보내는 것을 확인할 수 있음

■ 코드 검증 과정

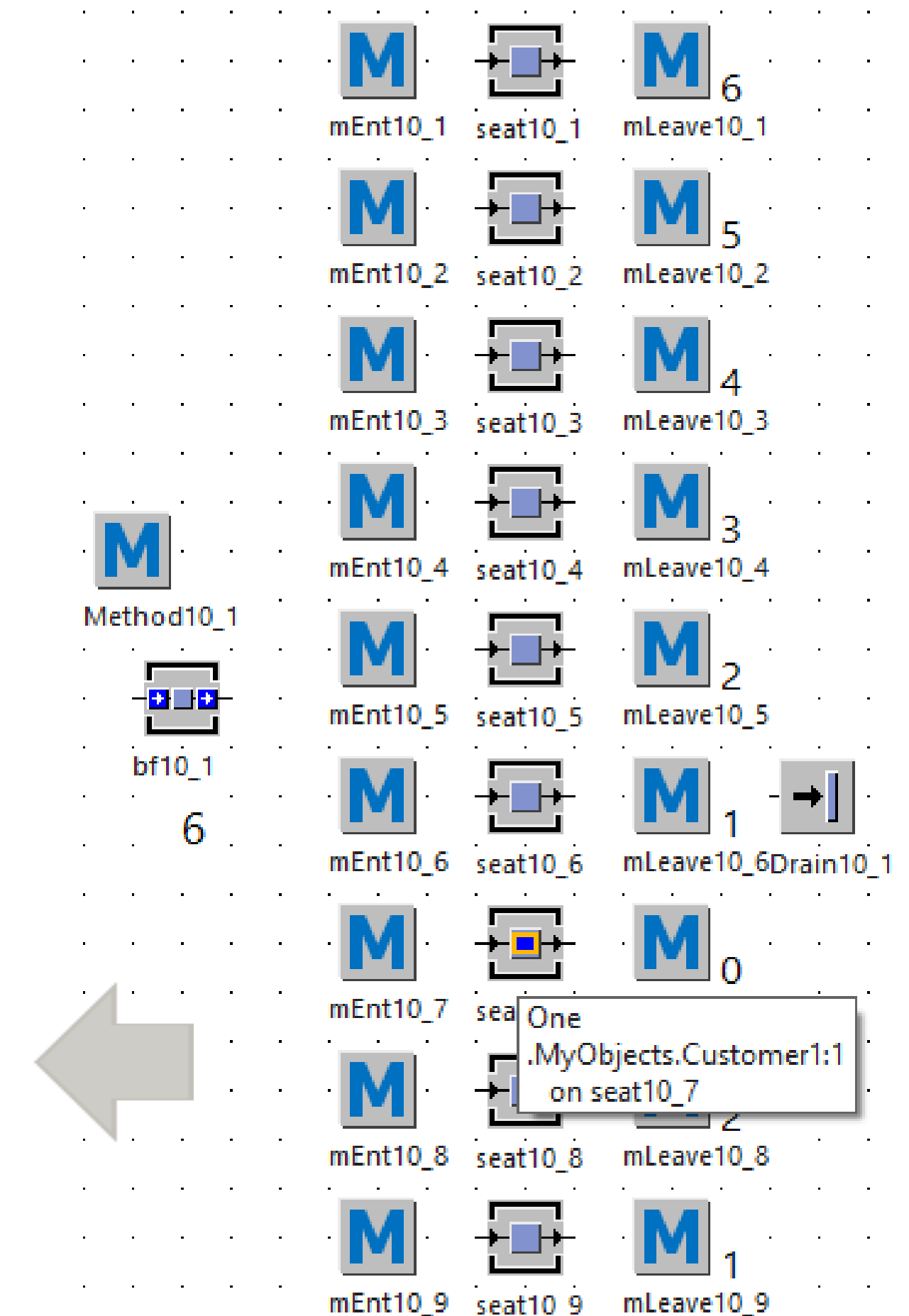


<시뮬레이션 Entity 생성기 및 좌석의 일부>

1. 특정 구역(x_x)의 capacity가 고객 수 수용 가능하다면,
그 구역에 무작위로 앉는지 확인

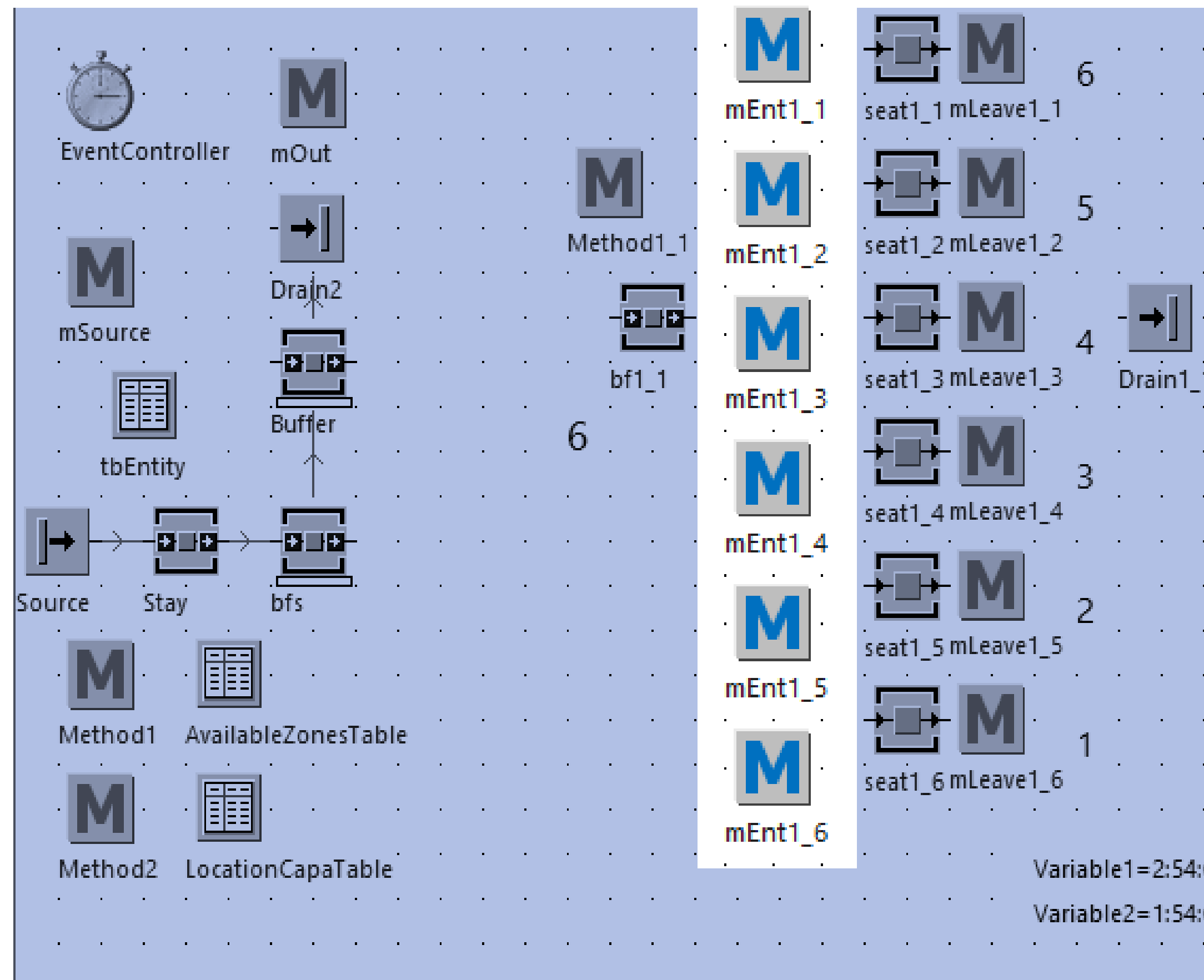


실험 시작하고 처음 생성된 Entity
(자리가 전부 비어있음)
→ **고객 수**와 **capacity**를 비교하여
특정구역을 정하고 무작위로 좌석을
점유하는 것을 확인할 수 있음



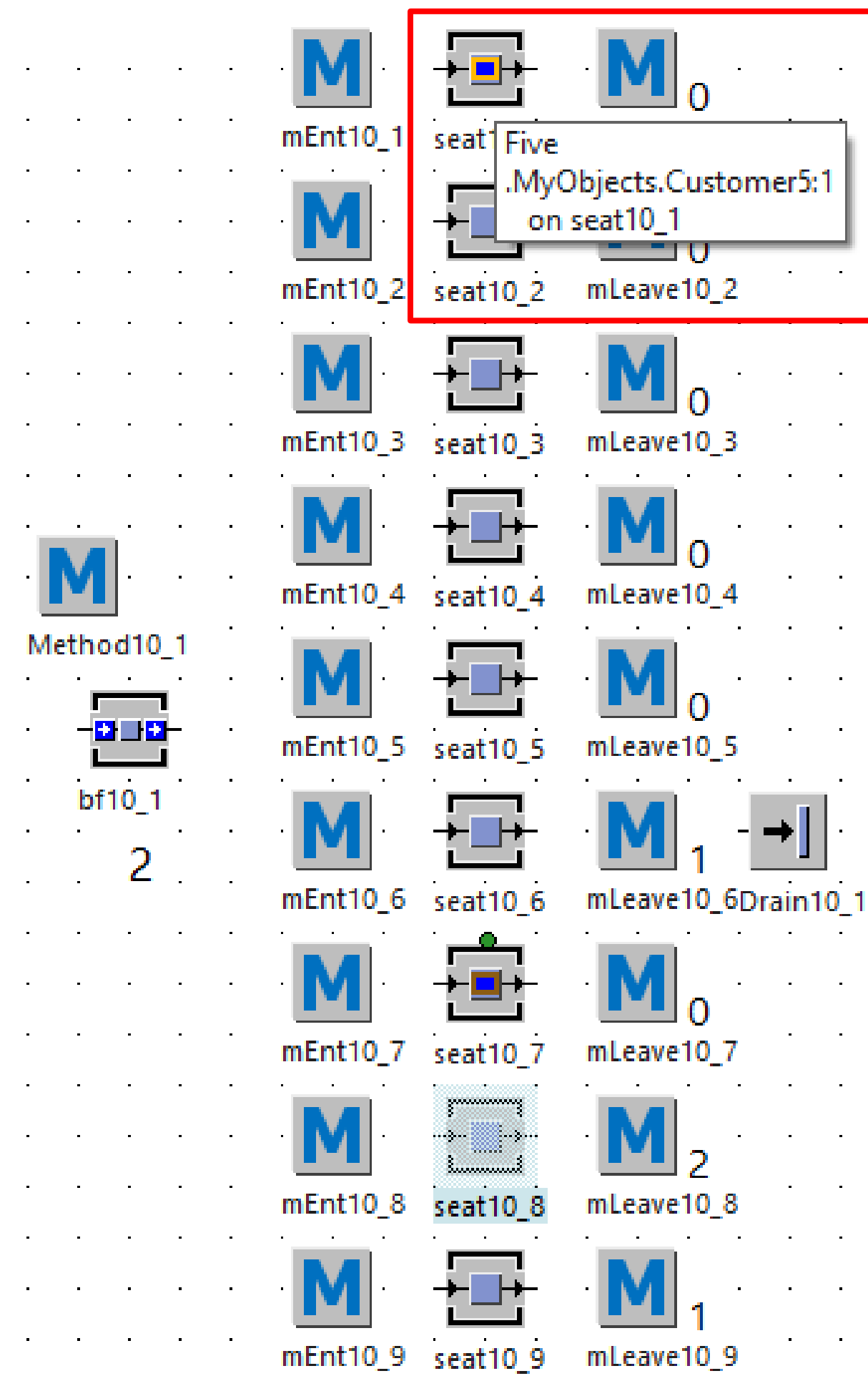
03-1. Verification – mEntx_x

■ 코드 검증 과정



<시뮬레이션 Entity 생성기 및 좌석의 일부>

Seat가 점유 되었을 때 각 자리의 capacity가 올바르게 계산이 되는지 확인



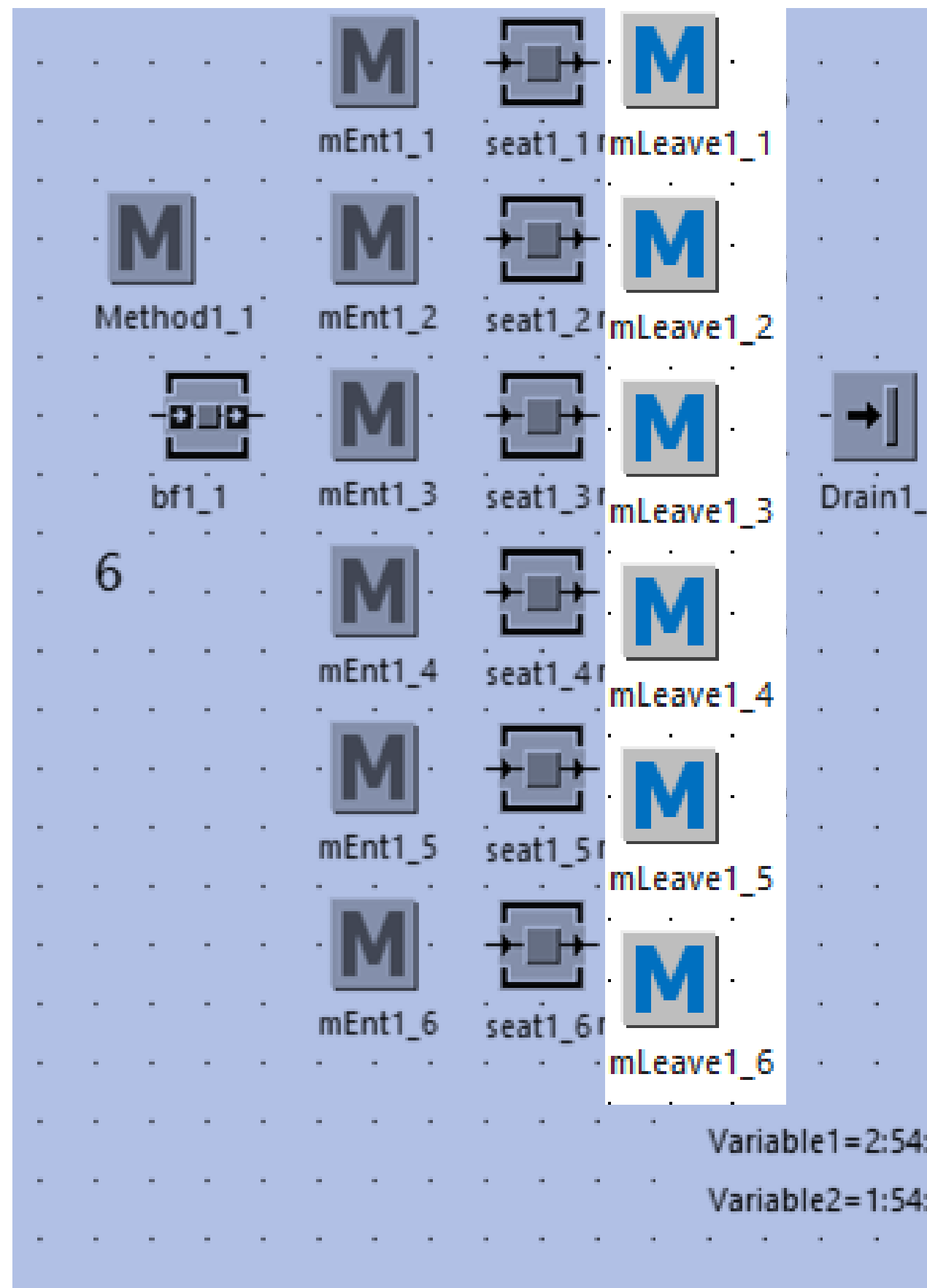
현재 5명의 손님이 들어왔으므로
아래로 5자리의 capacity
및 점유처리 상태의 업데이트가 필요



각 seat의
User-defined tab에서 확인가능

■ 코드 검증 과정

이전의 mEntx_x와 반대 로직으로 고객이 서비스이용 후 나갔을 때 시트의 점유 상태 및 용량이 원상복구 됐는지 검증



<시뮬레이션 좌석의 일부>

첫번째 좌석

Label: ☐ Failed ☐ Entrance locked ☐ Exit locked ☐ Planned

Name	Value	Type	C..	I..
aCapa	6	integer	*	
aCapaM	(Double-click to open)	method	*	
aEnter	1	integer	*	

두번째 좌석

Label: ☐ Failed ☐ Entrance locked ☐ Exit locked ☐ Planned

Name	Value	Type	C..	I..
aCapa	5	integer	*	
aCapaM	(Double-click to open)	method	*	
aEnter	1	integer	*	

세번째 좌석

Label: ☐ Failed ☐ Entrance locked ☐ Exit locked ☐ Planned

Name	Value	Type	C..	I..
aCapa	4	integer	*	
aCapaM	(Double-click to open)	method	*	
aEnter	1	integer	*	

네번째 좌석

Label: ☐ Failed ☐ Entrance locked ☐ Exit locked ☐ Planned

Name	Value	Type	C..	I..
aCapa	3	integer	*	
aCapaM	(Double-click to open)	method	*	
aEnter	1	integer	*	

다섯번째 좌석

Label: ☐ Failed ☐ Entrance locked ☐ Exit locked ☐ Planned

Name	Value	Type	C..	I..
aCapa	2	integer	*	
aCapaM	(Double-click to open)	method	*	
aEnter	1	integer	*	

위와 같이 5명 고객이 나갔을 때,
영향을 받은 5개의 좌석이 원래 상태의
aCapa와 **aEnter**로 잘 복구된 것을 확인할 수 있음

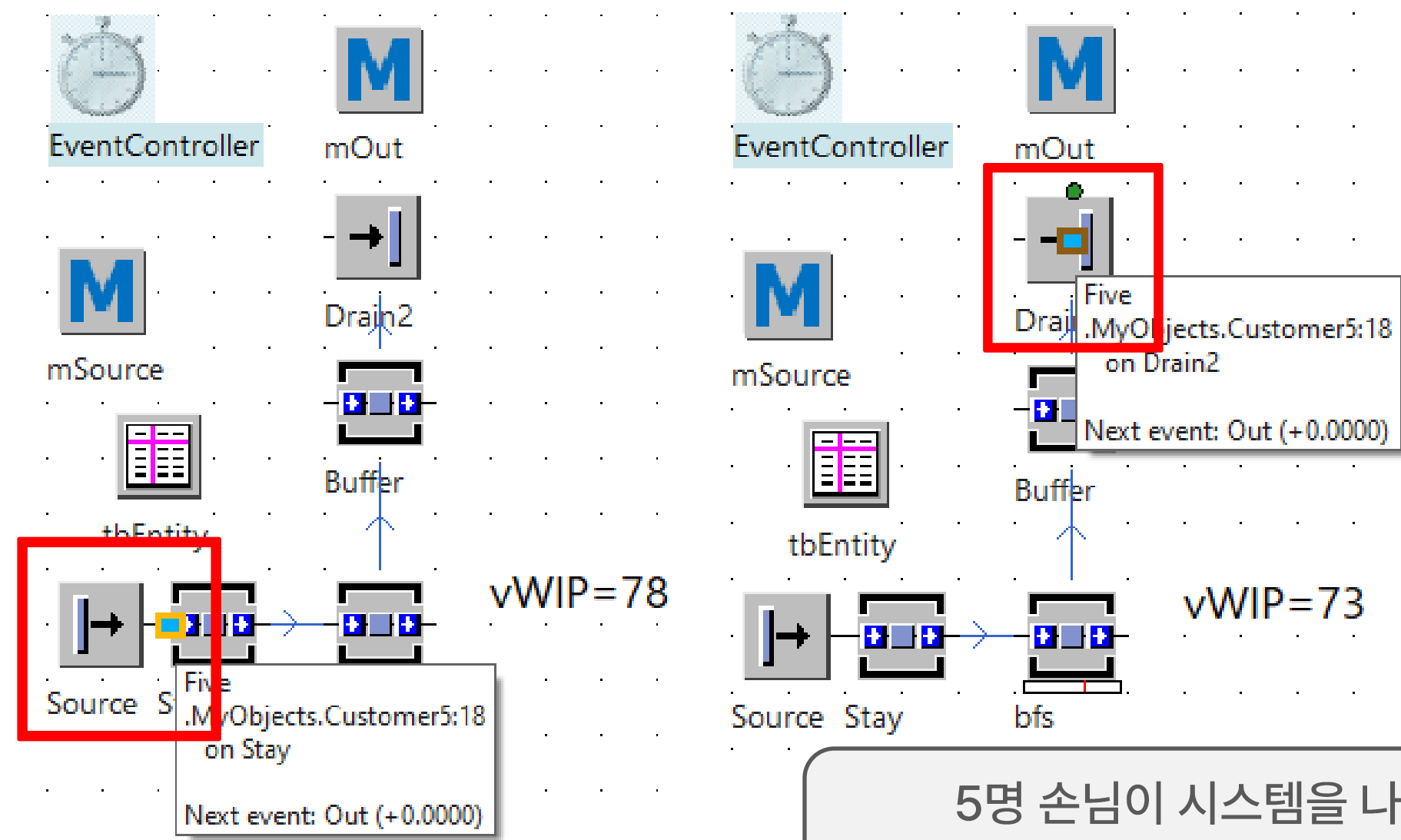
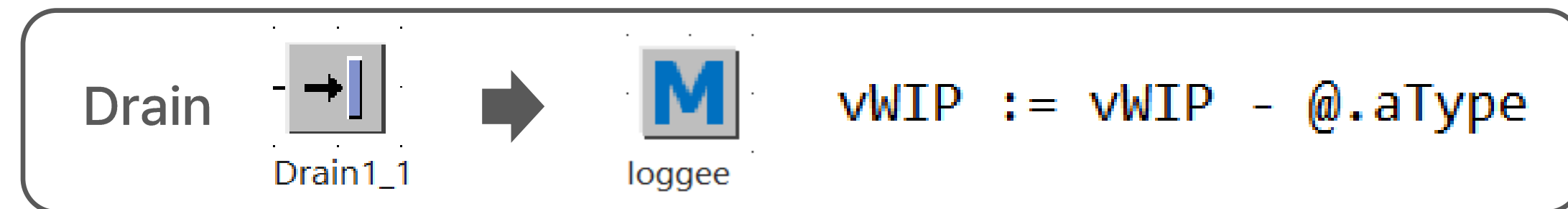
■ 시뮬레이션이 의도대로 구현됐는지 검증

1. WIP

- Source에서 entity가 생성될 때 추가



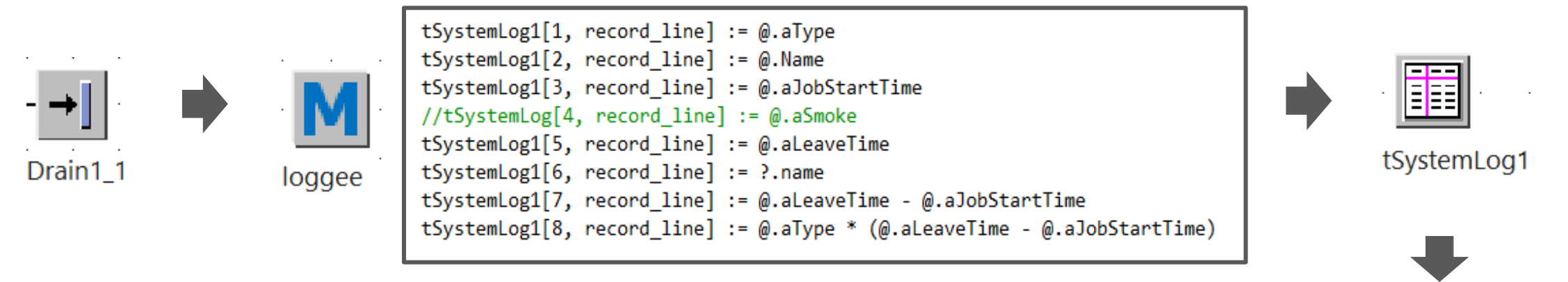
- Drain에서 entity가 퇴장할 때 감소



5명 손님이 시스템을 나가자
WIP: 78 → 73 으로 감소

2. table에 기록

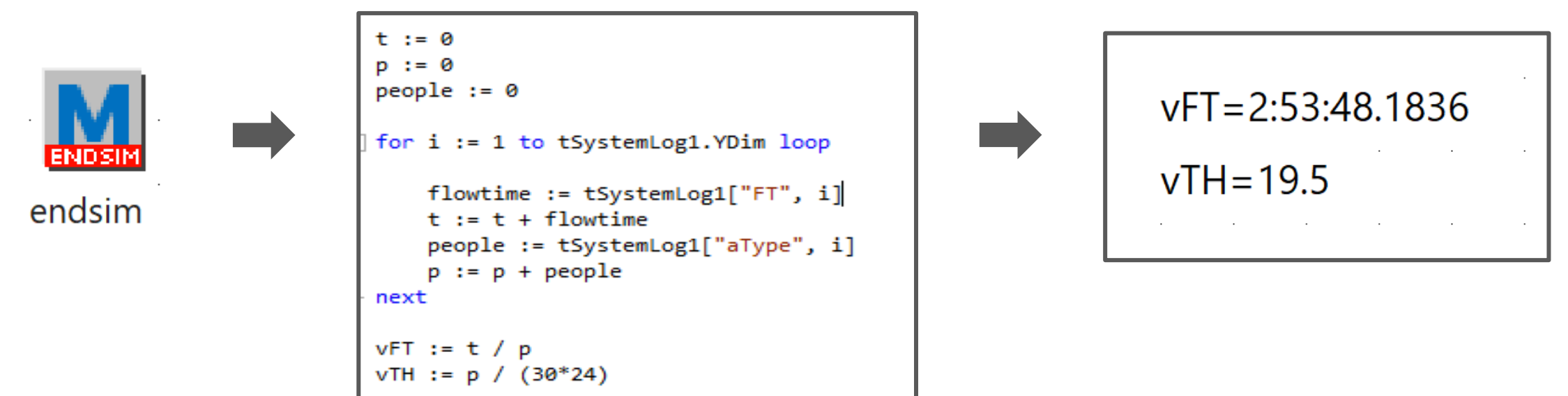
- 손님의 유형, 사용시작 시간, 사용종료 시간, 체류 시간 등을 기록



	integer 1	string 2	datetime 3	boolean 4	datetime 5	object 6	time 7	time 8
string	aType		StartTime		EndTime	Exit	체류시간	FT
1	4	Four	2022/01/03 06:28:07.5099		2022/01/03 07:10:11.4337	Drain7_2	42:03.9237	2:48:15.6949
2	5	Five	2022/01/03 06:01:16.2310		2022/01/03 07:17:08.6575	Drain10_1	1:15:52.4265	6:19:22.1327
3	1	One	2022/01/03 07:00:27.2236		2022/01/03 07:48:12.5827	Drain7_1	47:45.3591	47:45.3591
4	1	One	2022/01/03 06:00:00.0000		2022/01/03 08:21:15.7189	Drain10_1	2:21:15.7189	2:21:15.7189
5	2	Two	2022/01/03 06:44:22.3647		2022/01/03 08:31:24.7619	Drain10_2	1:47:02.3972	3:34:04.7944
6	3	Three	2022/01/03 06:42:29.7342		2022/01/03 09:13:32.6135	Drain5_1	2:31:02.8793	7:33:08.6379
7	2	Two	2022/01/03 07:47:02.8508		2022/01/03 09:24:02.5353	Drain10_1	1:36:59.6845	3:13:59.3691
8	4	Four	2022/01/03 07:24:28.6560		2022/01/03 09:39:51.6167	Drain9_2	2:15:22.9606	9:01:31.8425
9	4	Four	2022/01/03 08:10:52.6356		2022/01/03 09:50:50.1788	Drain6_2	1:39:57.5433	6:39:50.1731
10	5	Five	2022/01/03 09:19:12.9314		2022/01/03 09:59:56.7505	Drain7_2	40:43.8191	3:23:39.0956

3. table 결과를 바탕으로 평균 흐름시간과 시간당 처리량 계산

- 평균 흐름시간 (vFT), 시간당 처리량 (vTH)



■ WIP의 검정 과정

1. vWIP변수의 Statistics 기능으로 실제 WIP 계산

- $WIP = \sum(value_i \times Duration_i[\%])$

	Value	Frequency	Duration	Frequency [%]	Duration [%]	Mean Duration	Standard Deviation
1	21	1	7:55.6661	0.01	0.02	7:55.6661	0.0000
2	24	2	3:09.7353	0.02	0.01	1:34.8677	1:33.5224
3	25	1	1:26.2058	0.01	0.00	1:26.2058	0.0000
4	26	3	13:25.9858	0.03	0.03	4:28.6619	4:35.0832
5	27	4	12:01.1876	0.03	0.03	3:00.2969	3:16.7727
6	28	2	14:22.1327	0.02	0.03	7:11.0664	7:46.1214
7	29	6	30:33.2026	0.05	0.07	5:05.5338	4:12.2375
8	30	16	59:25.0872	0.13	0.14	3:42.8179	3:29.8657
9	31	15	38:16.6477	0.13	0.09	2:33.1098	2:17.3342
10	32	25	1:50:05.0784	0.21	0.26	4:24.2031	6:05.7844
11	33	28	1:37:12.0742	0.23	0.23	3:28.2884	2:55.4923
12	34	36	4:02:05.9464	0.30	0.57	6:43.4985	7:28.4275
13	35	45	3:16:20.7077	0.38	0.46	4:21.7935	5:01.8812
14	36	50	3:34:14.2335	0.42	0.50	4:17.0847	4:44.4723
15	37	65	5:58:11.1328	0.55	0.84	5:30.6328	6:36.5206
16	38	74	5:21:33.3493	0.62	0.75	4:20.7209	3:19.6189
17	39	94	6:20:32.7855	0.79	0.89	4:02.9020	4:17.5612
18	40	95	6:57:23.3176	0.80	0.98	4:23.6139	4:38.2926
19	41	117	8:23:26.5858	0.98	1.18	4:18.1759	3:48.9414
20	42	143	11:02:36.9405	1.20	1.56	4:38.0206	4:15.8206

	A	B	C	D	E	F	G	H	I	J
1	Value	Frequency	Duration	Frequency	Duration(%)	Mean Duration	Standard Deviation			
2	21	1	07:55.7	0.01	0.02	07:55.7	0			=A2*E2*0.01
3	24	2	03:09.7	0.02	0.01	01:34.9	01:33.5			0.0024
4	25	1	01:26.2	0.01	0	01:26.2	0			0
5	26	3	13:26.0	0.03	0.03	04:28.7	04:35.1			0.0078
6	27	4	12:01.2	0.03	0.03	03:00.3	03:16.8			0.0081
7	28	2	14:22.1	0.02	0.03	07:11.1	07:46.1			0.0084
8	29	6	30:33.2	0.05	0.07	05:05.5	04:12.2			0.0203
9	30	16	59:25.1	0.13	0.14	03:42.8	03:29.9			0.042
10					0.09					0.0279
WIP계산 =SUM(J2:J62)										WIP = 56.8438

2. 평균 흐름시간과 시간당 처리량 값으로 이론적 WIP 계산

■ Little's Law

$WIP = Flow\ Time \times Throughput$

$$WIP = vFT \times vTH$$
$$= 2.8967(hr) \times 19.5 \left(\frac{pc}{hr}\right)$$
$$= 56.48765$$

vFT=2:53:48.1836
vTH=19.5

3. 이론적 값과 실제 값의 비교

	이론적 WIP	실제 WIP
WIP 값	56.48756	56.8438

➡ 일치함을 확인 !

■ 기존 레이아웃 실험 결과

1. 기존 레이아웃 가정사항

- 나눠 앉을 확률: 0.22(22%), 시간당 요금: 1200원, 수요 발생 및 체류 시간 분포는 모든 실험에서 고정

“가격이 낮아지면 PC방에 머물고자 하는 의지가 상승할 것 → 나눠 앉을 의향 증가”

PC방의 1시간 요금이 1100원이라면, 단체(4~5명)로 방문했는데 모두 일렬로 앉을 수 없는 경우 어떻게 하나요?

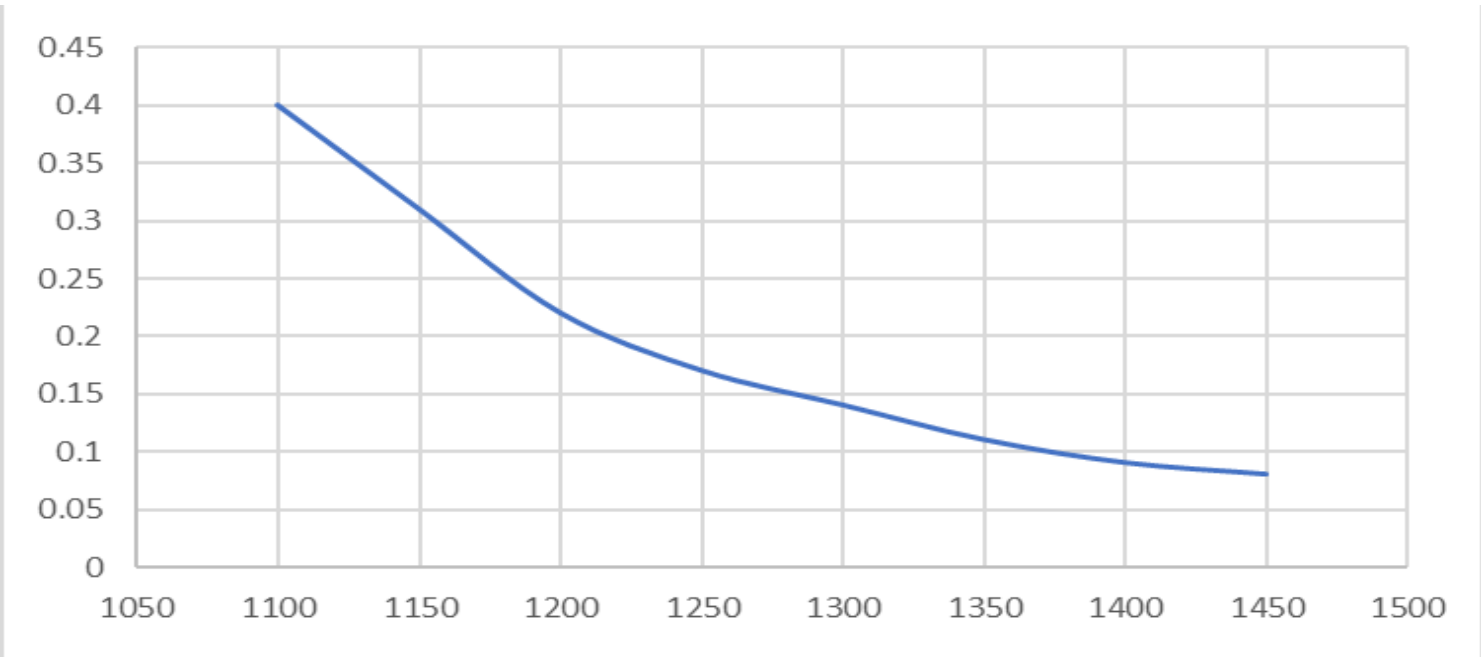
☐ 다른 PC방으로 간다

☐ 그냥 나눠 앉는다 ex) 5명의 경우(2명/3명)

PC방의 1시간 요금이 1150원이라면, 단체(4~5명)로 방문했는데 모두 일렬로 앉을 수 없는 경우 어떻게 하나요?

☐ 다른 PC방으로 간다

☐ 그냥 나눠 앉는다 ex) 5명의 경우(2명/3명)



시간당 비용	나눠 앉을 확률
1450	0.08
1400	0.09
1350	0.11
1300	0.14
1250	0.17
1200	0.22
1150	0.31
1100	0.4

<PC방 생산성 KPI>

1) 좌석 활용률

(이용된 좌석 시간 ÷ 전체 좌석 가능 시간) × 100

2) 고객 이탈률

(좌석 단가 × 이용된 좌석 시간)

3) 총 수익

(이탈한 총 고객 수 ÷ 전체 방문 고객 수) × 100

기존 레이아웃 실험 결과

2. 기존 레이아웃 KPI 분석

1) 좌석 활용률

좌석 활용률이 지속적으로 증가하지는 않으나,
유의하게 증가하는 부분을 확인
활용율의 증가가 목적이므로 특정 부분만 확인

	Exp 2	Exp 3	Exp 4	Exp 5	Exp 6	Exp 7	Exp 8
Exp 1	0.7	0.358	0.153	0.023	0.012	0	0
Exp 2		0.594	0.327	0.075	0.043	0	0
Exp 3			0.699	0.256	0.169	0.001	0
Exp 4				0.423	0.288	0.002	0
Exp 5					0.784	0.017	0
Exp 6						0.036	0
Exp 7							0.053

Experiment	좌석 활용률	표준편차
Exp1	0.526	0.008
Exp5	0.531	0.009
Exp7	0.537	0.008

2) 고객 이탈률

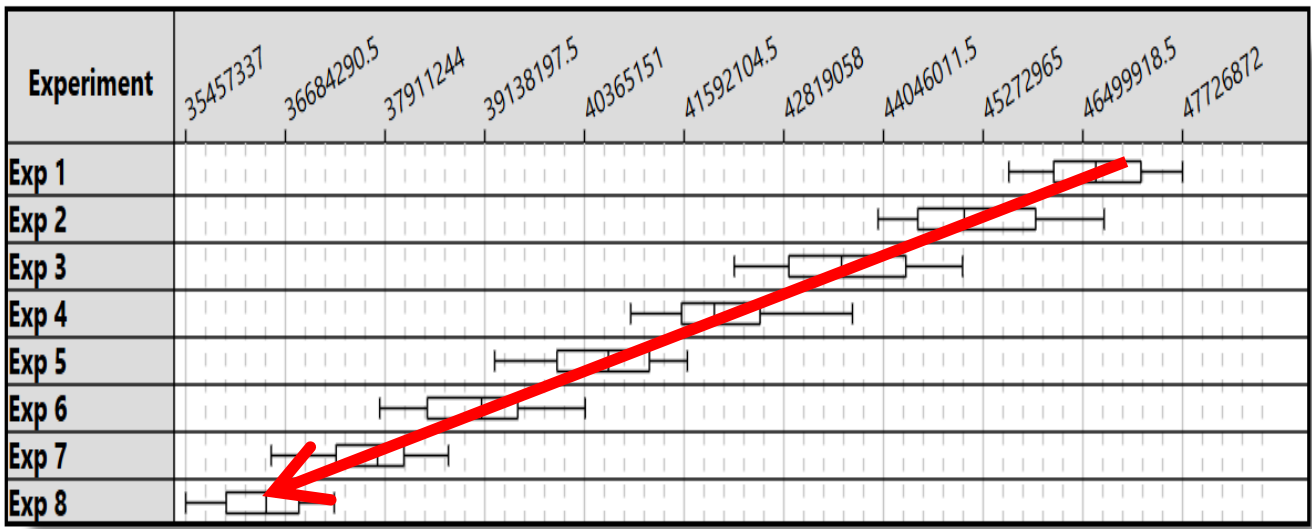
고객 이탈률이 지속적으로 감소하지는 않으나,
유의하게 감소하는 부분을 확인
고객 이탈률 감소가 목적이므로 특정 부분만 확인

	Exp 2	Exp 3	Exp 4	Exp 5	Exp 6	Exp 7	Exp 8
Exp 1	0.704	0.208	0.243	0.008	0	0	0
Exp 2		0.403	0.47	0.033	0.001	0	0
Exp 3			0.865	0.182	0.005	0	0
Exp 4				0.104	0.001	0	0
Exp 5					0.065	0	0
Exp 6						0.004	0
Exp 7							0.018

Experiment	고객 이탈률	표준편차
Exp1	0.048	0.006
Exp5	0.044	0.004
Exp7	0.039	0.004
Exp8	0.036	0.005

3) 총 수익

더 많은 수요를 확보하기 위해서
요금을 인하하여 고객 분할을 유도하였으나,
총 수익은 지속적으로 감소함을 관측!



Experiment	총 수익	표준편차
Exp1	46679411.7	669850
Exp8	3641087	560889

좌석 활용률↑, 고객 이탈률↓ → 총 수익↓

→ PC방의 목적: 총 수익 ↑ 이므로, 요금을 인하하여 고객 좌석 분할을 유도하는 전략은 최종 목적인 수익 증가에 도달하지 못함

→ 레이아웃 변경, 운영 정책 변화 등 다른 개선안의 구상이 필요

■ 비교 레이아웃 선정

1. 선정 배경

게임순위

2025년 10월 29일

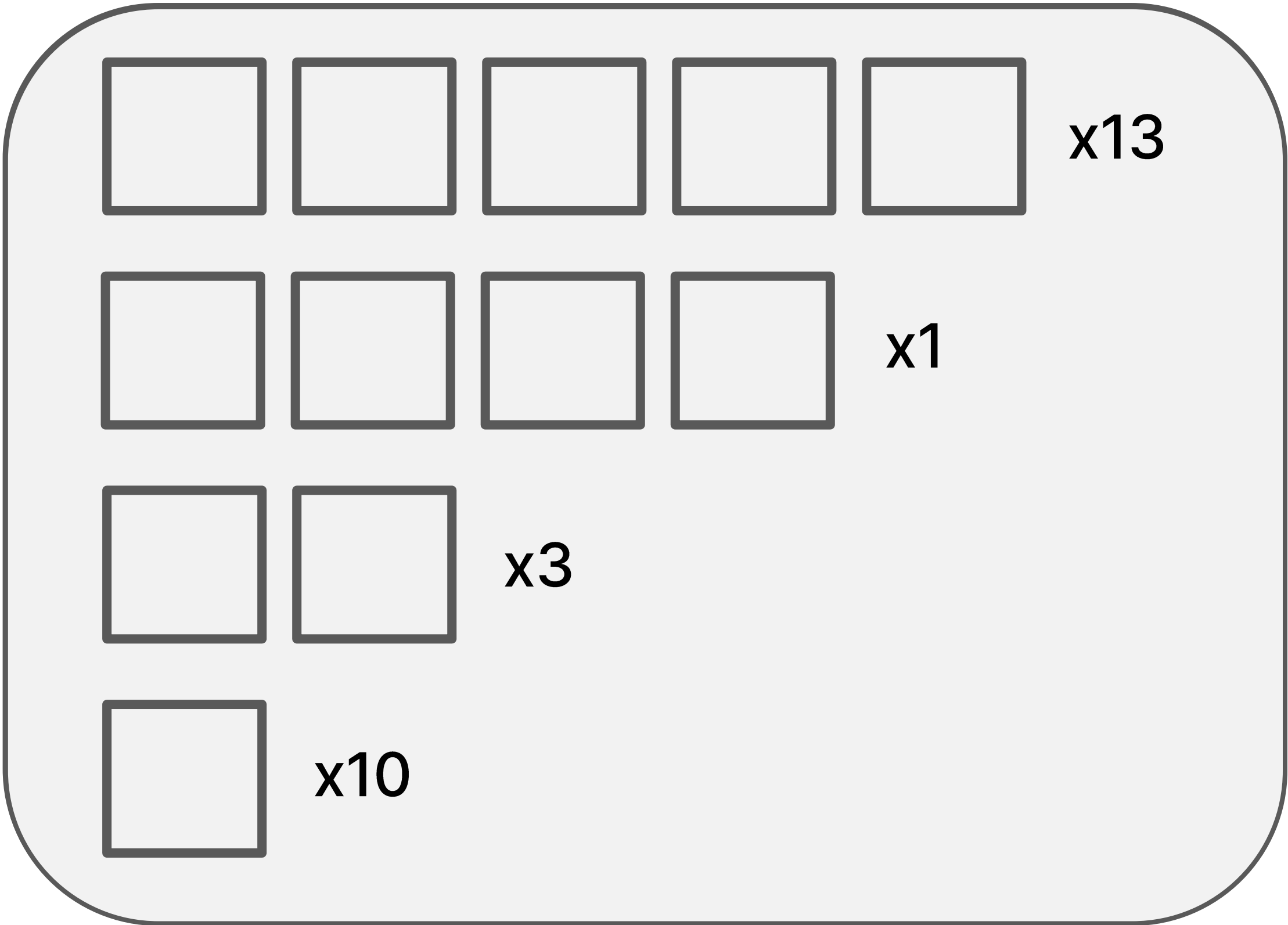
순위	게임정보	점유율
1	LEAGUE OF LEGENDS 378주 1위 라이엇 게임즈 라이엇 게임즈 리그 오브 레전드 38.44%	
2	배틀그라운드 10.31%	
3	발로란트 8.15%	
4	FC온라인 7.13%	
5	서든어택 4.94%	
6	오버워치 4.62%	
7	메이플 스토리 2.78%	
8	로스트아크 2.66%	
9	스타크래프트 2.15%	
10	Roblox 2.09%	

게임명	게임별 필요 인원수
리그 오브 레전드	5명
배틀그라운드	1/2/4명
발로란트	5명
FC온라인	모든 인원 가능
서든어택	1/5명
오버워치	5명

<게임 점유율 순위>

→ PC 점유율 상위권(1-6등) 게임들에 필요한 인원수를 고려하여
필요 인원수 별 점유율 비율을 구함

2. 선정 결과



→ 아테나 PC방의 총 좌석 수인 85석을 해당 비율로 분배
→ 5명 좌석(13개), 4명 좌석(1개), 2명 좌석(3개), 1명 좌석(10개)

■ 기존 레이아웃과 비교 레이아웃 생산성 비교

1. Values of Experiments

- 나눠 앉을 확률: 0.22(22%), 시간당 요금: 1200원, 수요 발생 및 체류 시간 분포는 모든 실험에서 고정 → 두 레이아웃에 공통 적용
- Input Values(모델 파라미터)
- Output Values(시뮬레이션 결과)

Input values	set by	Original Value	Inherited	Technical Notation
root.vDividedPer	Tab			root.vDividedPer
root.vCostperTime	Tab			root.vCostperTime

Target value	evaluated by	Technical Notation	High Values are Better
root.vSeatAvail	Tab	root.vSeatAvail	true
root.vTotalCost	Tab	root.vTotalCost	true
root.vExit_Customer_per	Tab	root.vExit_Customer_per	false

2. 좌석 활용률 비교

- 기존 레이아웃

Experiment	좌석 활용률	표준편차	Min 값	Max 값
기존 레이아웃	0.532	0.009	0.515	0.549

- 비교 레이아웃

Experiment	좌석 활용률	표준편차	Min 값	Max 값
기존 레이아웃	0.667	0.038	0.594	0.752

- T-검정 및 F-검정

유의미한 차이를 보임을 확인!

	T-stat	p-value_T	F-stat	p-value_F
좌석 활용률	-18.597	0	0.056	0

1) 좌석 활용률 증가
비교 레이아웃에서 좌석 활용률의 평균이 약 25.4% 증가함

2) 데이터 변동성 증가
비교 레이아웃의 표준편차가 약 4.2배 증가함 + 변동 범위 역시 확대됨
= 비교 레이아웃이 기존 레이아웃보다 활용률 값의 변동성이 크다!

■ 기존 레이아웃과 비교 레이아웃 생산성 비교

3. 총 수익 비교

- 기존 레이아웃

Experiment	총 수익	표준편차	Min 값	Max 값
기존 레이아웃	39,046,092	676,785	37,844,946	40,374,351

- 비교 레이아웃

Experiment	총 수익	표준편차	Min 값	Max 값
기존 레이아웃	48,987,819	2,848,702	43,588,192	55,197,890

4. 고객 이탈률 비교

- 기존 레이아웃

Experiment	이탈률	표준편차	Min 값	Max 값
기존 레이아웃	0.042	0.004	0.034	0.049

- 비교 레이아웃

Experiment	이탈률	표준편차	Min 값	Max 값
기존 레이아웃	0.06	0.006	0.049	0.071

- T-검정 및 F-검정

유의미한 차이를 보임을 확인!

	T-stat	p-value_T	F-stat	p-value_F
총 수익	-18.597	0	0.056	0

1) 총 수익 증가

비교 레이아웃에서 총 수익의 평균이 약 25.5% 증가함

2) 데이터 변동성 증가

비교 레이아웃의 표준편차가 약 4.2배 증가함 + 변동 범위 역시 확대됨
= 비교 레이아웃이 기존 레이아웃보다 총 수익 값의 변동성이 크다!

- T-검정 및 F-검정

유의미한 차이를 보임을 확인!

	T-stat	p-value_T	F-stat	p-value_F
고객 이탈률	-13.923	0	0.422	0.023

1) 고객 이탈률 증가

비교 레이아웃에서 고객 이탈률의 평균이 약 1.8% 증가함

→ **효율성**(높은 활용률/수익)과 **만족도**(고객 이탈률) 사이에 **Trade-off 관계**가 있음을 확인!

■ 기존 레이아웃 및 기존의 운영정책 분석

- 기존의 PC방이 운영되던 KPI를 이용하여 내부 생산성 변화를 분석

- 기존의 좌석활용률을 높이기 위해 실행되던 운영전략은 실제로 수익 감소로 귀결
- 따라서, 기존의 전략이 아닌 새로운 개선안 적용의 필요성 제기

경험 기반의 문제정의로 레이아웃 개선 및 분석

- 레이아웃 간 생산성 비교 분석

- 상호 레이아웃 간 동일한 조건으로 시뮬레이션을 시행 및 주요 결과 요약

- 1) 비교군의 좌석 활용률의 평균이 **약 25.4% 증가**
- 2) 이탈률의 평균이 **약 1.8% 증가**
- 3) 비교군에서 총 수익의 평균이 **약 25.5% 증가**

이탈률의 증가폭에 비해 활용률과 총 수익에서 큰 증가폭을 보이며 개선안이 타당함을 증명

05. 한계 및 향후 개선 방향

■ 현실성, 타당성 측면

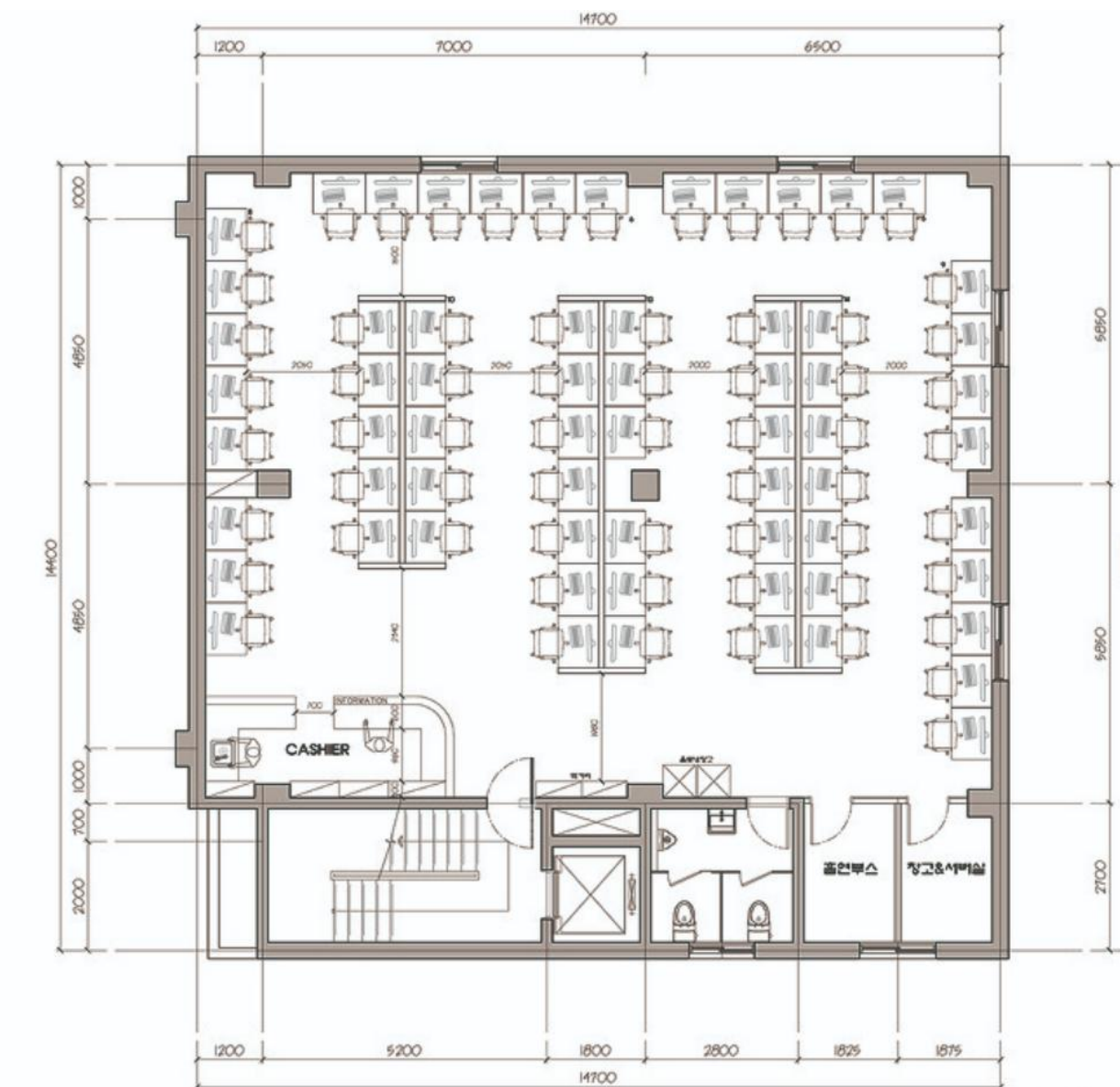
1. PC방 입장 시 모르는 사람이 있는 경우 한 자리 띄어 앉는 경향 **구현 X**
2. 대학가 앞이라는 **특정 상황**에서만 시뮬레이션 진행
3. 음식 주문, 점원과의 거리, 실제 PC방 크기 **고려 X**

■ 향후 계획

1. 설문조사, 현장조사를 통해 **고객들의 추가적인 행동 패턴** 수집 및 구현
2. 주거지역, 학군지와 같은 **여러가지 수요 상황** 조사 및 구현
3. **실제 PC방의 크기**를 고려하여 여러가지 레이아웃 간 생산성 비교 분석 예정
4. 음식을 주문 시 **추가 수익 발생** 고려 및 생산성 분석 반영
5. 레이아웃 간 점원의 **서빙 이동거리 고려** 및 음식 **제공까지 걸리는 평균 시간** 측정예정
6. **모듈화**를 통해 타 PC방에서 해당 시뮬레이션 사용 용이성 증대



<모르는 사람끼리 떨어져 앉은 모습>

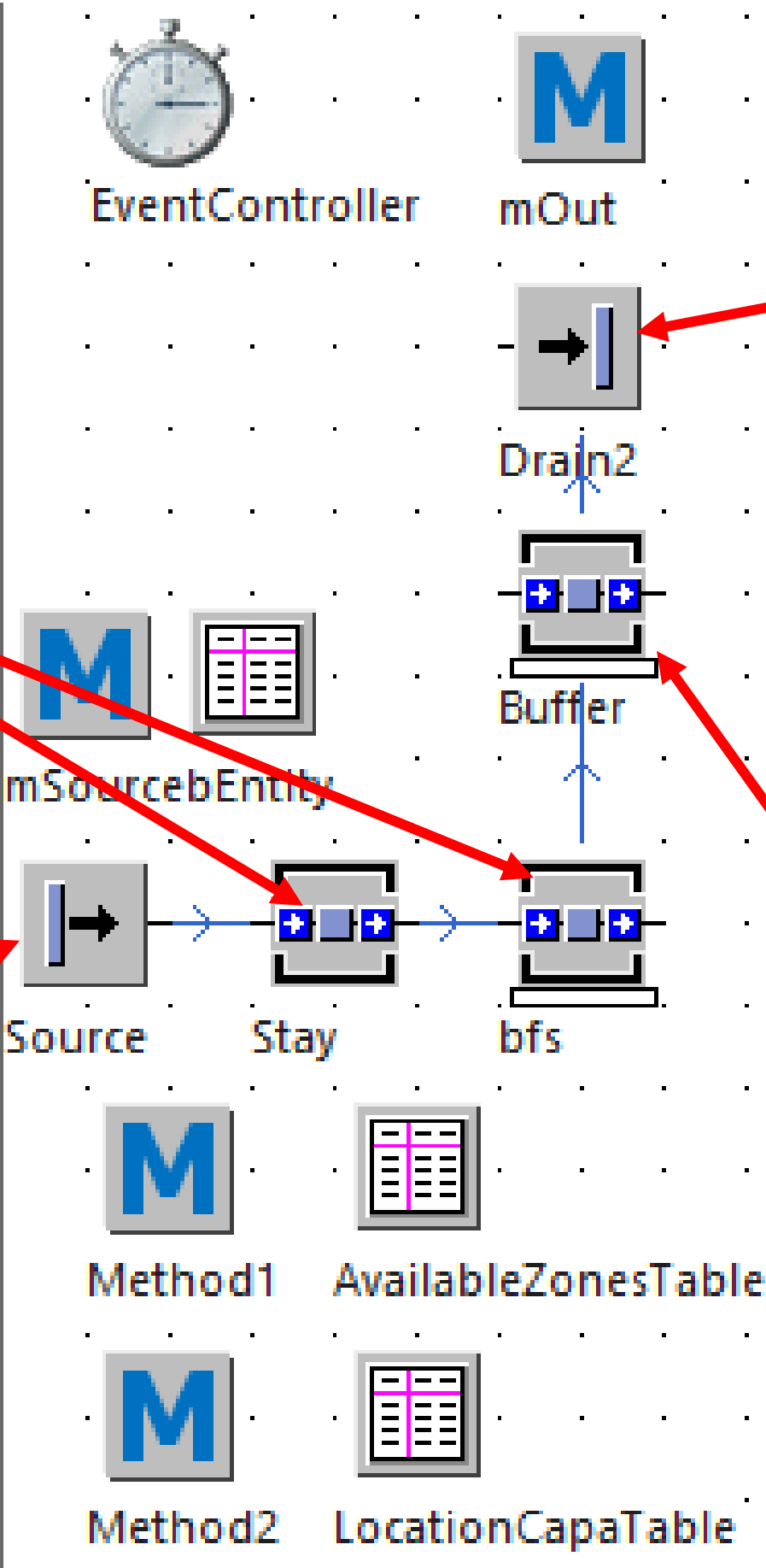


<건물 크기를 고려한 PC방 설계도>

① 입구 - 손님 발생

손님 배치 계산 용이를 위한 버퍼

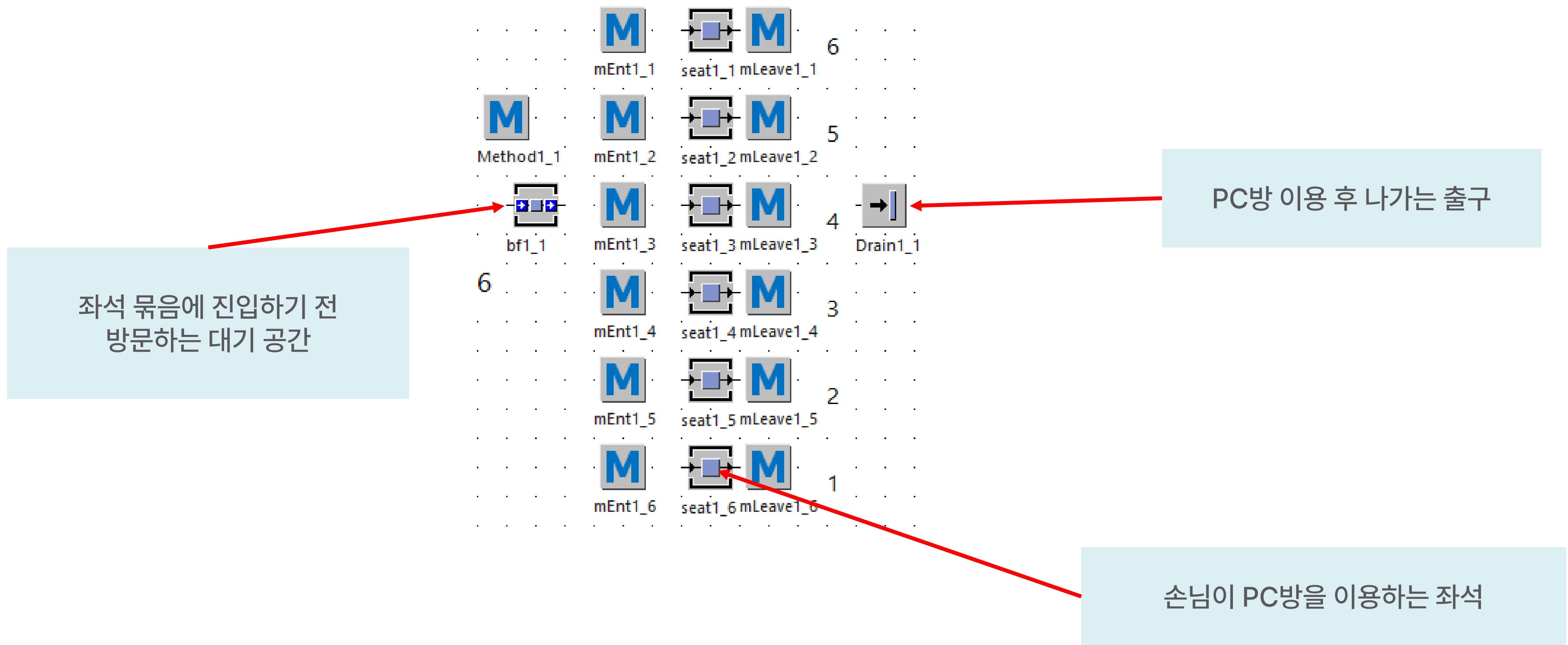
자리 부족으로 이탈하는 손님 출구



손님 발생(픽크 시간 12시~22시 고려)

서비스 받을 손님 대기 공간(시간 측정 x)

② 좌석 – PC방 이용



■ 엔티티(고객) 생성 및 도착 간격 정의



mSource

```
if @.aType = 4 or @.aType = 5 then
  r := z_uniform(1, 0.0, 1.0)
  if r < vDividedPer
    @.aDivide := True
  end
end
```

```
CurrentTime_Sec := EventController.SimTime
```

```
PeakStart_Sec := 12 * 3600
```

```
PeakEnd_Sec := 22 * 3600
```

```
if (CurrentTime_Sec >= PeakStart_Sec) and (CurrentTime_Sec < PeakEnd_Sec) then
```

```
  NextArrivalTime := z_negexp(04:06.26)
```

```
else
```

```
  NextArrivalTime := z_lognorm(8:52.87, 5:23.92)
```

```
end
```

```
Source.Interval := NextArrivalTime
```

<목적>

1. 나뉘질 의향이 있는 고객을 설문 조사 기반해 결정한다
2. 현재 시각을 기준으로 피크 시간대를 구분하여 다음 고객이 도착할 간격을 각각 데이터에 기반한 분포로 계산 후 도착 간격을 시스템에 반영

1. 나뉘질 의향이 있는 고객

- 1) 고객의 수가 4명 이상일때, 0부터 1사이의 임의의 실수 추출
- 2) 그 값이 vDividedPer보다 작으면 나뉘질 의향이 있는 고객으로 결정

2. 피크 시간대를 구분해서, Interval Time 설정

- 1) 현재 시간을 확인해서, 현재 시간이 Peak 시작 시간과 PeakEnd_Sec 끝 시간 사이라면 지수 분포로 고객 도착
- 2) 그 외의 시간에는 로그정규분포로 고객 도착

■ 손님 이용 가능 구역 배정



Method1

```
AvailableZonesTable.delete

MUCapacity := @.aType
--5

for i := 1 to LocationCapaTable.YDim loop

    v_Zone := LocationCapaTable[1, i]
    v_MaxCapa := LocationCapaTable[2, i]

    if v_MaxCapa >= MUCapacity then

        AvailableZonesTable[1, AvailableZonesTable.YDim+1] := v_Zone
    end

next

v_AvailableCount := AvailableZonesTable.YDim

if v_AvailableCount > 0 then

    v_Choice := z_uniform(1, v_AvailableCount)
    TargetZone := AvailableZonesTable[1, v_Choice]

    @.move(TargetZone)

else

    @.move(Buffer)

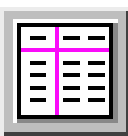
end
```

<목적>

- 1. 무작위로 구역을 배정해주기보다는 먼저 가능한 구역을 찾는다.
- 2. 그 구역에서 자리를 찾아가도록 한다.

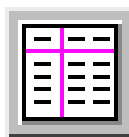
1. LocationCapaTable에서 모든 구역의 수용 가능 인원을 확인하고, **수용 가능한 인원이 손님의 수보다 크다면** 갈 수 있는 구역으로 정해서 AvailableZonesTable에 넣어준다

2. 이후에 가능한 구역에 랜덤하게 손님을 이동시킨다
만약 갈 수 있는 구역이 없다면 PC방 나가기 전 버퍼로 이동시킨다



LocationCapaTable

	object 1	integer 2
1	bf1_1	5
2	bf2_1	5
3	bf3_1	5
4	bf4_1	5
5	bf5_1	5
6	bf7_1	4
7	bf8_1	2
8	bf9_1	2
9	bf11_2	5
10	bf6_2	5
11	bf7_2	5
12	bf8_2	5
13	bf9_2	5
14	bf10_2	5
15	bf12_2	5
16	bf13_2	5
17	bf11	1
18	bf22	1



AvailableZonesTable

	object 1
1	*.Models.model_cut.bf2_1
2	*.Models.model_cut.bf3_1
3	*.Models.model_cut.bf5_1
4	*.Models.model_cut.bf9_1
5	*.Models.model_cut.bf8_2
6	*.Models.model_cut.bf9_2
7	*.Models.model_cut.bf10_2
8	*.Models.model_cut.bf13_2
9	
10	

■ 단체손님 나눠 앉기 & 손님 이탈



mOut

```
for i := 1 to LocationCapaTable.YDim loop
  --v_Zone := LocationCapaTable[1, i]
  v_MaxCapa := LocationCapaTable[2, i]
  if v_MaxCapa >= 2 then
    count2 := count2 + 1
  end
  if v_MaxCapa >= 3 then
    count3 := count3 + 1
  end
end
next

if @.aType = 4 or @.aType = 5 then
  if @.aDivide then
    if @.aType = 4 then
      if count2 >= 2 then
        local newMU : object := .MyObjects.Customer4_2.create(bfs)
        local newMU1 : object := .MyObjects.Customer4_2.create(bfs)

        local gametime : real := z_lognorm(Variable1, Variable2)

        newMU.aProcTime := gametime;
        newMU1.aProcTime := gametime;

        if @.aSmoke then
          newMU.aSmoke := True
          newMU1.aSmoke := True
        end
        @.delete
      else
        @.move(Drain2)
      end
    end
  end
end
```

<목적>

4,5인 손님이 앉을 수 있는 좌석이 없을 때,
나눠질 의향이 있는 4,5인 손님의 경우 나눠서 앉을 수 있게 한다.

- 1) LocationCapaTable에서 각 구역의 수용 가능한 인원이 2명 이상인 개수와 3명 이상인 개수를 각각 count2, count3에 각각 저장한다
- 2) 만약 해당 손님이 나눠질 의향이 있는 4인 손님인 경우 2인 손님 2개로 나뉘고, 5인 손님은 2인 손님과 3인 손님으로 나뉜다
- 3) 각각에 같은 ProcTime을 배정하고, 기존 손님 Entity를 제거한다
- 4) 단체 손님(4, 5인)이 아닌 경우, 나뉘 의향이 없는 손님인 경우 그리고 나눠질 수 없는 경우에 손님은 이탈한다(Drain2)

■ 구역에 들어온 손님 좌석에 할당



Method7_1

```
?.MaxCapaM

while not moved
  r := z_uniform(1, 0.0, 1.0)

  if r < 0.25 and seat7_1.aEnter = 1 then
    @.move(seat7_1)
    moved := true
  elseif r < 0.50 and seat7_2.aEnter = 1 then
    @.move(seat7_2)
    moved := true
  elseif r < 0.75 and seat7_3.aEnter = 1 then
    @.move(seat7_3)
    moved := true
  elseif r <= 1.0 and seat7_4.aEnter = 1 then
    @.move(seat7_4)
    moved := true
  end
end

?.MaxCapaM
Method2
```



mEnt7_2

```
if ?.aEnter = 0 then
  @.move(bf7_1)

elseif @.aType > ?.aCapa then
  @.move(bf7_1)

elseif ?.occupied then
  ?.aEnter := 0
  ?.aCapa := 0
  if @.hasAttribute("aProcTime") then
    ?.ProcTime := @.aProcTime
  else
    ?.ProcTime := z_lognorm(Variable1, Variable2)
  end

  if @.aType = 2 then
    seat7_3.aEnter := 0
    seat7_3.aCapa := 0

  elseif @.aType = 3 then
    seat7_3.aEnter := 0
    seat7_4.aEnter := 0
    seat7_3.aCapa := 0
    seat7_4.aCapa := 0
  end

  seat7_1.aCapaM

end

@.aJobStartTime := EventController.AbsSimTime
```

<목적>

1. 구역에 들어온 손님을 대표 좌석에 배치한다.
2. 서비스 시간을 할당
3. 손님 수에 따라서 종속 좌석을 할당

- 1) 구역에 들어온 손님을 랜덤한 좌석에 할당
- 2) 출입 가능하고(aEnter = 1), 수용 가능 인원보다 손님수가 많으면 재할당 시도
- 3) 할당이 된 경우 대표 좌석 사용불가 처리, 단체손님(2~5인) 경우 종속좌석 사용불가 처리
- 4) 모든 좌석의 수용 가능 인원 재계산 (aCapaM)

■ 손님 퇴장



mLeave7_2

```
if ?.aEnter = 0 then
  ?.aEnter := 1
  if @.aType = 2 then
    seat7_3.aEnter := 1
    seat7_3.aCapaM
  end
  if @.aType = 3 then
    seat7_3.aEnter := 1
    seat7_4.aEnter := 1
    seat7_4.aCapaM
    seat7_3.aCapaM
  end
  ?.aCapaM
  seat7_1.aCapaM
end
bf7_1.MaxCapaM
@.move(Drain7_1)
```

<목적>

사용 마친 손님이 좌석을 떠날 때, 좌석 사용 가능 처리

- 1) 손님이 사용을 마치고 좌석을 떠날 때, 대표 좌석을 사용 가능 처리
- 2) 단체손님인 경우, 종속 좌석도 사용 가능 처리 함
- 3) 모든 좌석의 수용 가능 인원 재계산 (aCapaM)
- 4) 구역에 최대 수용 가능 인원 재계산, 손님 퇴장 (Drain)

■ [기존 레이아웃 실험] 시간당 요금: 1200원, 나눠 앓을 확률: 0.1 ~ 0.9

1) 좌석 활용률

나눠 앓을 확률에 비례해서 좌석 활용률이 증가하고, 유의함을 확인

	Exp 2	Exp 3	Exp 4	Exp 5	Exp 6	Exp 7	Exp 8	Exp 9
Exp 1	0.497	0.003	0	0	0	0	0	0
Exp 2		0.019	0	0	0	0	0	0
Exp 3			0.01	0	0	0	0	0
Exp 4				0.041	0	0	0	0
Exp 5					0.003	0	0	0
Exp 6						0.048	0	0
Exp 7							0.01	0
Exp 8								0.001

Experiment	좌석 활용률	표준편차
Exp1	0.528	0.009
Exp4	0.541	0.009
Exp6	0.551	0.008

2) 총 수익

총 수익이 나눠 앓을 확률에 비례해 증가하고, 유의함을 확인

	Exp 2	Exp 3	Exp 4	Exp 5	Exp 6	Exp 7	Exp 8	Exp 9
Exp 1	0.497	0.003	0	0	0	0	0	0
Exp 2		0.019	0	0	0	0	0	0
Exp 3			0.01	0	0	0	0	0
Exp 4				0.041	0	0	0	0
Exp 5					0.003	0	0	0
Exp 6						0.048	0	0
Exp 7							0.01	0
Exp 8								0.001

Experiment	총 수익	표준편차
Exp1	38795486	635299
Exp4	39295743	611885
Exp6	40466142	561975

3) 이탈률

나눠 앓을 확률이 증가하면, 이탈률도 감소하고, 유의함

	Exp 2	Exp 3	Exp 4	Exp 5	Exp 6	Exp 7	Exp 8	Exp 9
Exp 1	0.174	0	0	0	0	0	0	0
Exp 2		0.003	0	0	0	0	0	0
Exp 3			0.004	0	0	0	0	0
Exp 4				0.001	0	0	0	0
Exp 5					0	0	0	0
Exp 6						0.001	0	0
Exp 7							0	0
Exp 8								0

Experiment	이탈률	표준편차
Exp1	0.0458	0.0053
Exp9	0.0143	0.0028

1) WIP

나눠 앓을 확률에 비례해 WIP이 전체적으로 유의하게 감소하진 않음
일부 실험에선 유의함

	Exp 2	Exp 3	Exp 4	Exp 5	Exp 6	Exp 7	Exp 8	Exp 9
Exp 1	1	0.438	0.973	0.713	0.726	0.159	0.066	0.01
Exp 2		0.415	0.971	0.701	0.714	0.143	0.056	0.007
Exp 3			0.402	0.698	0.688	0.452	0.219	0.048
Exp 4				0.708	0.722	0.13	0.048	0.004
Exp 5					0.988	0.288	0.132	0.026
Exp 6						0.284	0.13	0.026
Exp 7							0.642	0.295
Exp 8								0.605

Experiment	WIP	표준편차
Exp1	45.3	8.367
Exp8	48.7	9.622
Exp9	50.8	7.621

MaxCapaM

```
var maxcapa, maxcapa1, maxcapa2, maxcapa3 : integer

maxcapa3 := max(seat4.aCapa, seat5.aCapa)
maxcapa2 := max(seat3.aCapa, maxcapa3)
maxcapa1 := max(seat2.aCapa, maxcapa2)
maxcapa := max(seat1.aCapa, maxcapa1)

self.~.MaxCapa := maxcapa
```

aCapaM

```
self.~.aCapa := 0
if self.~.aEnter = 1 then
  self.~.aCapa += 1
  if seat5_3.aEnter = 1 then
    self.~.aCapa += 1
    if seat5_4.aEnter = 1 then
      self.~.aCapa += 1
      if seat5_5.aEnter = 1 then
        self.~.aCapa += 1
      end
    end
  end
end
end
end
```

■ 좌석 입구 버퍼에 최대 수용 인원 확인

<목적>

손님을 소스부터 좌석 입구 버퍼에 분배하기 위해, 이 구역의 최대 수용 가능 인원을 확인

- 1) 모든 좌석의 최대 수용 인원을 확인해서 Max 값을 속성으로 가짐

■ 좌석 사용 가능 여부 확인, 좌석 최대 수용 인원

<목적>

연결된 손님 진입 가능 여부를 확인해서, 현재 좌석에 진입 가능한 최대 손님 수 확인

- 1) 현재 내 좌석 아래 연결된 좌석들이 하나 증가 될 때마다, aCapa에 1씩 늘리면서 진입 가능 최대 손님을 확인함

- Table에 좌석 입구 버퍼에 최대 수용 인원 입력



Method1

```
var i : integer

for i := 1 to LocationCapaTable.YDim loop
    LocationCapaTable[2,i] := LocationCapaTable[1,i].MaxCapa
next
```

<목적>

모든 좌석 입구 버퍼에 최대 수용 인원을 LocationCapaTable에

갱신
모든 좌석 입구 버퍼의 각 MaxCapa 값을 확인해
LocationCapaTable에 재갱신